

PENN: ALGO TRADING WEB PLATFORM

A PROJECT REPORT

*Submitted in partial fulfillment of the
Requirements for the award of the degree*

Of

Bachelor of Technology in Information Technology

By

Himela Biswas

(IT/22/30, University Roll no:10700222063)

Rupayan Panja

(IT/22/41 , University Roll no:10700222038)

Nabin Maity

(IT/22/39, University Roll no: 10700222035)



DEPARTMENT OF INFORMATION TECHNOLOGY

COLLEGE OF ENGINEERING & MANAGEMENT, KOLAGHAT

**(AFFILIATED TO MAULANA ABUL KALAM AZAD UNIVERSITY OF TECHNOLOGY,
WEST BENGAL)**



2025-26



DEPARTMENT OF INFORMATION TECHNOLOGY
COLLEGE OF ENGINEERING & MANAGEMENT, KOLAGHAT
(Affiliated to MAKAUT, WB)
Purba Medinipur – 721171, West Bengal, India

CANDIDATE’S DECLARATION

We hereby declare that the work presented in the final year project entitled: “**PENN: ALGO TRADING WEB PLATFORM**” submitted in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Information Technology**, to the **Department of Information Technology, College of Engineering & Management, Kolaghat**, is an authentic record of my/our own work carried out during the final year under the guidance of: **Dr. Apurba Roy, Assistant Professor , Information Technology Department.**

We further declare that the contents of this report have not been submitted to any other university or institution for the award of any degree, diploma, fellowship, or any other similar title.

Himela Biswas
University Roll No.: 10700222063
University Registration No.:221070110297

Rupayan Panja
University Roll No.: 10700222038
University Registration No.:221070110313

Nabin Chandra Maity
University Roll No.: 10700222038
University Registration No.:221070110301



DEPARTMENT OF INFORMATION TECHNOLOGY
COLLEGE OF ENGINEERING & MANAGEMENT, KOLAGHAT
(Affiliated to MAKAUT, WB)

Purba Medinipur – 721171, West Bengal, India

CERTIFICATE OF APPROVAL

This is to certify that the work embodied in this project entitled: “**PENN: Algo Trading Web Platform**” submitted by:

Himela Biswas (10700222063)

Rupayan Panja (10700222038)

Nabin Chandra Maity (10700222035)

To the Department of Information Technology, has been carried out under my supervision and guidance.

The project work has been prepared in accordance with the regulations of Maulana Abul Kalam Azad University of Technology (MAKAUT). I hereby recommend that this project report be accepted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Information Technology.

Supervisor
Dr. Apurba Roy
Dept. of Information Technology

Dr. Apurba Roy
Head, Department of Information Technology



DEPARTMENT OF INFORMATION TECHNOLOGY
COLLEGE OF ENGINEERING & MANAGEMENT, KOLAGHAT
(Affiliated to MAKAUT, WB)

Purba Medinipur – 721171, West Bengal, India

CERTIFICATE BY THE BOARD OF EXAMINERS

This is to certify that the project work entitled: “ **PENN: Algo Trading Web Platform** ” submitted by

Himela Biswas (10700222063)

Rupayan Panja (10700222038)

Nabin Chandra Maity(10700222035)

To the Department of Information Technology, College of Engineering & Management, Kolaghat, has been examined and evaluated by the undersigned.

The project work has been prepared in accordance with the regulations of Maulana Abul Kalam Azad University of Technology (MAKAUT) and is hereby approved as a partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Information Technology.

Project Coordinator

Board of Examiners

ACKNOWLEDGEMENT

It gives us immense pleasure to express our deep and sincere gratitude to our project guide, Dr. Apurba Roy, for his invaluable guidance, constant encouragement, and continuous support throughout the course of this project. His insightful suggestions, constructive feedback, and meticulous supervision have greatly shaped the direction and quality of our work. We are sincerely thankful to the Department of Information Technology, College of Engineering & Management, Kolaghat, for providing us with the necessary facilities and a supportive academic environment that enabled us to carry out our project successfully.

We also extend our heartfelt thanks to all the faculty members, laboratory staff, library staff, and administrative personnel of the institute for their kind assistance and cooperation during the project period.

Lastly, we would like to express our warm appreciation to our fellow classmates, whose encouragement and helpful suggestions provided additional motivation and direction throughout this endeavor.

Himela Biswas
University Roll No.: 10700222063
University Registration No.: 221070110297

Rupayan Panja
University Roll No.: 10700222038
University Registration No.: 221070110313

Nabin Chandra Maity
University Roll No.: 10700222038
University Registration No.: 221070110301

Abstract

This project, titled **“PENN: Algo Trading Web Platform”**, presents the design and development of a comprehensive web-based algorithmic trading platform that integrates historical market analysis, intelligent strategy recommendation, and real-time market monitoring. The system aims to address the limitations of manual trading, including emotional decision-making, inefficient strategy evaluation, and the lack of accessible analytical tools, by providing an automated and data-driven trading environment.

The platform is developed using a modern full-stack architecture, with Next.js and TypeScript for the frontend, Flask for the backend, and PostgreSQL (Supabase) for secure cloud-based data storage. Historical stock market data is collected through the Yahoo Finance API, while live market information is obtained using the Fyers API. The system implements twenty-three algorithmic trading strategies across multiple categories, including trend following, momentum, breakout, volatility, mean reversion, and statistical arbitrage. Each strategy is evaluated using a historical backtesting engine that generates key performance metrics such as return, win rate, Sharpe ratio, maximum drawdown, and overall risk.

To enhance decision-making, the platform incorporates a Machine Learning Recommendation Engine based on the Random Forest algorithm. The recommendation engine analyzes the performance metrics of all implemented strategies, applies a scoring mechanism, and suggests the most suitable strategy for the selected stock while providing transparent performance insights. Interactive visualizations, including Plotly-based candlestick charts, strategy comparisons, and analytical dashboards, enable users to evaluate trading performance effectively.

The developed system successfully satisfies the functional requirements of an intelligent trading platform by integrating data processing, algorithmic trading, machine learning, secure authentication, and cloud database management into a unified application. The platform provides a scalable foundation for future enhancements, including automated trade execution, advanced portfolio management, and deployment in real-time trading environments.

Table of Contents

Candidate's declaration	2
Certificate of approval.....	3
Certificate by the board of examiners	4
Acknowledgements	5
Abstract	6
Table of Contents	7
List of Figures	9
List of Acronyms.....	9

Chapter 1

An introduction [11-29]

1.1 Introduction	11-12
1.2 Information processing.....	13-17
1.3 Financial Market and Trading System	17
1.4 Evolution and concept of algo trading	18
1.5 Role of Information Technology and Computing in Trading	18-19
1.6 Need for Automation, Data Processing and Decision Making	19-20
1.7 Educational ,practical and industry significance of the project	20-21
1.8 Objective and Organization of the report	22-29

Chapter 2

Review of Past Work and Problem Formulation [30-35]

2.1 Overview	30
2.2 Types of Algorithm Trading Strategies.	30-31
2.3 Review of Existing Algo Trading System	32
2.4 Limitations of existing System	32-34
2.5 Problem Formulation	34
2.6 Summary of Chapter	35

Chapter 3

System Analysis and requirements [36-43]

3.1 Introduction.....	36
3.2 Problem Definition	36
3.3 Existing System	37
3.4 Proposed System	37
3.5 Objective of the proposed System.....	37-38
3.6 Functional Requirement.....	38-42
3.7 Feasibility Study	42
3.8 Summary.....	43

Chapter 4

System design [44-48]

4.1 Introduction.....	44
4.2 System Architecture.....	44-45
4.3 Architectural Design Description	45
4.4 Data Flow Diagram (DFD).....	45-48
4.5 ER Diagram	48

Chapter 5

Algorithm and Trading Strategies [49-44]

5.1 Introduction.....	49
5.2 Role of Algorithms in Algorithmic Trading	49
5.3 Types of Trading Strategies	50
5.4 Algorithm Design and Pseudocode.....	51

5.5 Strategy Execution Flow.....	51-52
5.6 Backtesting Methodology	52
5.7 Live/Paper Trading Methodology.....	52-53
5.8 AI Strategy Recommendation Methodology.....	53-54
5.9 Performance Metrics.....	54
5.10 Advantages of Implemented Strategies.....	54-55
5.11 Limitation.....	55
5.12 Summary	55

Chapter 6

[56-68]

Methodology

6.1 Introduction.....	56-57
6.2 Tools and Technologies Used.....	57-59
6.3 Methodology Overview.....	59-61
6.4 Data Collection and Preparation.....	61-62
6.5 Implementation of Trading Algorithms.....	62
6.6 Machine Learning Recommendation Engine.....	62-64
6.7 Backtest and Performance Evaluation.....	64-65
6.8 User Interface Implementation.....	65-66
6.9 Integration of Module.....	66-67
6.9 Testing and Validation.....	67-68

6.10 Summary	68
--------------------	----

Chapter 7

Results and Discussion/Analysis [69-77]

7.1 Overview of Experimental Setup	69
7.2 Backtesting Results	69-70
7.3 Live Trading/ Paper Trading Result	70-72
7.4 AI Strategy Recommendation Result	72-73
7.5 User Profile (My Profile) Result	74-75
7.6 Comparison of Existing and Proposed System	75
7.7 Performance Discussion	76
7.8 Overall Summary of Result	76-77

Chapter 8

8.1 Conclusion	78
8.2 Scope for Future Work	79

References & Bibliography.....79-80

List of Figures

Figure No.	Description
Fig 01	DFD level 0
Fig 02	DFD level 1
Fig 03	DFD level 2 Backtest Module
Fig 04	DFD level 2 live Trading Module
Fig 05	ER Diagram
Fig 06	Hand Written Risk Calculation
Fig 07	Paper Trading/ Live Trading Setup
Fig 08	Backtesting Set up
Fig 09	Paper Trading Result with Built in Search Recommendation
Fig 10	Trading Result with Built in Search Recommendation
Fig 11	Recommendation engine
Fig 13	User Page (My Profile)

List of Acronyms

Acronym	Full Form
API	Application Programming Interface
ATR	Average True Range
CAGR	Compound Annual Growth Rate
CSV	Comma-Separated Values
DFD	Data Flow Diagram
ERD	Entity Relationship Diagram
HLC	High, Low, Close
MACD	Moving Average Convergence Divergence
ML	Machine Learning
OHLC	Open, High, Low, Close
P&L	Profit and Loss
RSI	Relative Strength Index
SMA	Simple Moving Average
EMA	Exponential Moving Average
API	Application Programming Interface
NSE	National Stock Exchange
ROI	Return on Investment
SQL	Structured Query Language

Chapter- 1

1.1 Introduction

In recent years, the financial markets have undergone a significant transformation driven by rapid advancements in information technology, data analytics, and high-speed internet connectivity. Trading, which was once dominated by manual processes and human judgment, has evolved into a technology-centric activity where automated systems and algorithms play a vital role. This shift has led to the emergence of **algorithmic trading**, a method that uses computer programs and predefined rules to execute trading decisions automatically.

Algorithmic trading, commonly referred to as *algo trading*, involves the use of mathematical models, statistical techniques, and technical indicators to identify trading opportunities and execute buy or sell orders without continuous human intervention. These systems are capable of processing large volumes of market data in real time and reacting to market changes at a speed and accuracy that is not possible for human traders. As a result, algorithmic trading helps reduce delays in execution, minimize human errors, and eliminate emotional biases that often affect manual trading decisions.

Traditionally, algorithmic trading was accessible only to large financial institutions, hedge funds, and professional traders due to high infrastructure costs and technical complexity. However, the widespread availability of open-source programming languages such as Python, cloud-based platforms, and market data APIs has significantly lowered the entry barrier. Today, students, researchers, and retail traders can experiment with algorithmic trading strategies using affordable and accessible tools. This democratization of algorithmic trading has increased the demand for simple, transparent, and user-friendly platforms that support learning and experimentation.

Despite its growing popularity, many existing algorithmic trading platforms present challenges for beginners. Commercial solutions often involve high subscription costs, complex interfaces, or black-box systems where users have limited visibility into how strategies operate. Beginners may find it difficult to understand strategy logic, evaluate performance, or test ideas safely before applying them in real markets. These challenges highlight the need for an educational and analytical platform that simplifies algorithmic trading while maintaining essential functionality.

The **PENN: Algorithmic Trading Web Platform** proposed in this project aims to address these challenges by providing a web-based environment that allows users to analyze market data, apply rule-based trading strategies, and evaluate their performance through systematic backtesting. The platform focuses on usability, transparency, and analytical clarity, enabling users to understand how trading decisions are generated rather than relying on opaque systems.

A key aspect of algorithmic trading is **backtesting**, which involves testing a trading strategy on historical market data to evaluate its effectiveness before deploying it in live markets. Backtesting helps traders understand the potential profitability, risk, and limitations of a strategy under different market conditions. By incorporating backtesting features, the proposed platform encourages informed and risk-aware decision-making, which is especially important for beginners.

The project also emphasizes **data visualization**, as graphical representation of price movements, indicators, and performance metrics plays a crucial role in interpreting trading results. Clear visual insights help users identify trends, assess strategy behavior, and make data-driven conclusions. Through interactive charts and tables, the platform enhances the overall analytical experience.

From an academic perspective, this project serves as a practical application of concepts from programming, data analysis, and financial markets. It bridges the gap between theoretical knowledge and real-world implementation by demonstrating how algorithms can be used to solve complex financial problems. The project also provides hands-on experience in working with APIs, handling real-time and historical data, and developing web-based analytical tools.

In summary, the Algorithmic Trading Web Platform is designed as an educational and analytical system that introduces users to the fundamentals of automated trading. By combining market data analysis, rule-based strategies, backtesting, and visualization in a single web-based application, the project aims to simplify algorithmic trading and make it accessible to a wider audience. This introduction sets the foundation for understanding the objectives, design, and implementation of the system discussed in the subsequent chapters of this report.

1.2 Information Processing

Information processing plays a crucial role in modern computing systems, especially in applications that deal with large volumes of data and real-time decision-making such as algorithmic trading platforms. In the context of this project, information processing involves the collection, computation, transformation, storage, and retrieval of financial market data to generate meaningful insights and automated trading decisions. Efficient information processing ensures accuracy, speed, reliability, and scalability of the system.

The Algorithmic Trading Web Platform relies heavily on information processing techniques to analyze historical and real-time market data, apply trading algorithms, perform backtesting, and visualize results. This chapter discusses the major components of information processing relevant to the project, namely Computing, Processing, and Database Systems.

1.2.1 Computing

Computing refers to the use of computational resources, hardware, and software to perform calculations, data analysis, and decision-making tasks. In algorithmic trading systems, computing forms the foundation upon which all analytical and operational activities are built.

Modern computing systems enable high-speed execution of complex algorithms, allowing trading strategies to respond instantly to market changes. The performance of an algorithmic trading platform depends significantly on the efficiency of its computing environment, including processor speed, memory management, and software optimization.

In this project, computing is primarily achieved through software-based computation using high-level programming languages and libraries. Python is used as the core programming language due to its simplicity, flexibility, and extensive support for data analysis and financial computation. Libraries such as Pandas and NumPy facilitate numerical computation and data manipulation, while visualization libraries assist in graphical analysis.

The computing process involves executing predefined algorithms that analyze price movements, calculate indicators, and generate trading signals. These computations must be accurate and optimized to handle large datasets efficiently. Even though this project is

designed mainly for educational and analytical purposes, the principles of efficient computing remain essential to ensure system responsiveness and correctness.

Furthermore, computing also includes handling real-time data streams received through APIs. The system must compute indicators dynamically as new data arrives, making computing a continuous and iterative process rather than a one-time operation.

1.2.2 Computing (Advanced)

Application-level computing focuses on how computational logic is applied to solve real-world problems. In the Algorithmic Trading Web Platform, this involves implementing financial models, trading strategies, and analytical workflows that transform raw data into actionable insights.

At this level, computing is concerned with defining algorithmic rules such as entry and exit conditions, indicator thresholds, and risk parameters. For example, a moving average crossover strategy requires computing short-term and long-term averages, comparing their values, and generating buy or sell signals accordingly. These computations must be performed repeatedly over historical datasets during backtesting and in near real time during analysis.

Another important aspect of application-level computing is modularity. The system is designed with separate modules for data fetching, strategy execution, backtesting, and visualization. This modular computing approach improves maintainability, scalability, and clarity of the system.

Computing also plays a significant role in performance evaluation. Metrics such as total return, win ratio, drawdown, and volatility are computed based on strategy results. These computed metrics help users assess the effectiveness and risk profile of trading strategies.

In addition, the computing environment must support error handling and validation to ensure system reliability. Invalid data, API failures, or logical errors in strategies must be identified and handled gracefully to avoid incorrect outputs.

Overall, application-level computing transforms theoretical trading concepts into practical, executable systems, making it a core pillar of the project.

1.2.3 Processing

Processing refers to the systematic transformation of raw data into meaningful information through a sequence of operations. In algorithmic trading, processing involves several stages, including data collection, cleaning, transformation, analysis, and output generation.

The first stage of processing is **data acquisition**, where market data such as stock prices, volume, and timestamps are collected from external APIs. This data is often received in raw or semi-structured formats and may contain missing values or inconsistencies.

The next stage is **data preprocessing**, which involves cleaning and organizing the data. This includes handling missing values, converting data types, aligning timestamps, and filtering relevant fields. Proper preprocessing is essential to ensure the accuracy of subsequent computations.

After preprocessing, the data undergoes **analytical processing**, where trading algorithms and indicators are applied. Technical indicators such as moving averages, momentum indicators, or trend signals are calculated during this stage. This processed information forms the basis for trading decisions and strategy evaluation.

Another critical aspect of processing is **backtesting**, where historical data is processed sequentially to simulate how a trading strategy would have performed in past market conditions. Backtesting allows users to evaluate profitability, consistency, and risk before applying strategies in live markets.

Finally, the processed data is converted into **visual and tabular outputs**. Charts, graphs, and performance tables help users interpret results effectively. Visualization is an essential part of processing, as it converts numerical outputs into intuitive insights.

Thus, processing acts as the bridge between raw market data and meaningful decision-making in the algorithmic trading platform.

1.2.4 Database System

A database system is responsible for storing, managing, and retrieving data efficiently. In an algorithmic trading platform, database systems play a crucial role in handling historical data, processed results, and analytical outputs.

The **Algorithmic Trading Web Platform** utilizes **PostgreSQL** hosted on **Supabase** as its primary database management system. This cloud-based relational database provides secure, reliable, and scalable storage while enabling efficient access to application data through structured queries and backend services. It supports persistent storage and seamless integration with the Flask backend.

The database system ensures data consistency, integrity, and accessibility throughout the application. User accounts, authentication details, portfolios, strategy information, and backtesting results are stored in an organized manner, allowing quick retrieval whenever required. This structured approach improves application performance and enables efficient management of trading-related information.

From a conceptual perspective, the database follows relational database principles such as normalization, data integrity, secure transactions, and efficient querying. PostgreSQL provides robust support for structured data management while Supabase offers cloud hosting, authentication support, and reliable database accessibility, making it suitable for both academic and practical applications.

Data security is another important consideration. Sensitive information such as user credentials, JWT authentication data, and API configuration details are managed securely through backend validation and database access controls. Proper security mechanisms ensure that stored information remains protected against unauthorized access.

In future versions, the database system can be further extended to support advanced portfolio analytics, real-time trading records, transaction history, and large-scale financial datasets. However, even in its current implementation, the PostgreSQL database provides a stable, secure, and scalable foundation for efficient data management within the platform.

1.2.5 Summary

Information processing is a fundamental component of the Algorithmic Trading Web Platform. Through efficient computing, structured processing, and reliable data storage mechanisms, the system transforms raw market data into valuable insights. Each component—computing, processing, and database systems—works together to ensure accuracy, speed, and usability. This structured approach to information processing enables the platform to serve as a practical and educational tool for understanding algorithmic trading systems.

1.3 Financial Market And Trading Systems

Financial markets play a crucial role in the economic development of any country by enabling the efficient allocation of capital and providing opportunities for investment and wealth creation. A financial market is a platform where buyers and sellers participate in the trading of financial instruments such as stocks, bonds, commodities, currencies, and derivatives. These markets facilitate liquidity, price discovery, and risk management for investors and institutions.

Stock markets such as the Bombay Stock Exchange (BSE) and National Stock Exchange (NSE) in India, along with global exchanges like NYSE and NASDAQ, operate as organized marketplaces where securities are traded under regulated conditions. Over time, financial markets have evolved from physical trading floors to fully electronic systems. This transformation has significantly improved transparency, accessibility, and trading efficiency.

Trading systems are the technological and procedural frameworks that support market operations. Early trading systems were manual in nature, relying heavily on human traders, physical order books, and verbal communication. Such systems were slow, prone to errors, and highly dependent on human judgment. With the advancement of technology, electronic trading systems were introduced, allowing orders to be placed and executed through computers and digital networks.

Modern trading systems rely on real-time data feeds, automated order matching, and high-speed communication networks. These systems process massive volumes of transactions every second and ensure fair and efficient market functioning. However, the complexity and

speed of modern markets have also increased the challenges faced by human traders, creating the need for automated solutions.

In this context, understanding financial markets and trading systems is essential for developing algorithmic trading platforms. The proposed project operates within this ecosystem by utilizing market data generated by these systems and applying computational techniques to analyze and interpret trading information.

1.4 Evolution And Concept of Algorithmic Trading

Algorithmic trading represents a significant milestone in the evolution of financial markets. It refers to the use of computer algorithms to automate trading decisions based on predefined rules and mathematical models. These algorithms determine when to buy or sell a financial instrument by analyzing market data such as price, volume, and historical trends.

The concept of algorithmic trading emerged with the growth of electronic exchanges and increased availability of digital market data. Initially, automated systems were used mainly for order execution to reduce transaction costs and minimize market impact. Over time, these systems evolved into sophisticated trading strategies capable of identifying complex patterns and executing trades autonomously.

Algorithmic trading eliminates many limitations of manual trading, such as emotional bias, delayed reactions, and limited data processing capability. By following predefined rules consistently, algorithms ensure discipline and objectivity in trading decisions. This consistency is especially important in volatile markets where emotions can lead to irrational decisions.

Different types of algorithmic trading strategies exist, including trend-following strategies, arbitrage strategies, and mean-reversion strategies. While advanced strategies may involve machine learning and artificial intelligence, rule-based strategies remain widely used due to their simplicity and transparency.

This project focuses on implementing fundamental algorithmic trading concepts in a web-based environment. By allowing users to test and analyze strategies through backtesting, the platform provides a practical understanding of how algorithmic trading works in real-world scenarios.

1.5 Role of Information Technology And Computing In Trading

Information technology is the backbone of modern financial markets and trading systems. The ability to collect, process, store, and analyse vast amounts of data in real time has transformed how trading activities are conducted. Computing power plays a critical role in enabling algorithmic trading systems to operate efficiently and accurately.

High-speed processors, optimized software, and reliable networks allow trading algorithms to analyse market data and execute orders within milliseconds. Programming languages such as Python have become popular in the financial domain due to their flexibility, readability, and extensive library support for data analysis and visualization.

Market data APIs provide structured access to real-time and historical data, enabling developers to build analytical tools and trading platforms. These APIs act as a bridge between stock exchanges and trading applications, making data-driven decision-making possible.

In addition to computation, information technology supports data visualization, system monitoring, and user interaction through web-based interfaces. Technologies such as Streamlit allow developers to create interactive dashboards that present complex financial data in an understandable format.

The Algorithmic Trading Web Platform developed in this project leverages these technologies to perform data analysis, strategy execution, and visualization. The effective use of computing resources ensures that the platform delivers accurate results and a responsive user experience.

1.6 Need For Automation, Data Processing, And Decision Making

The increasing complexity and speed of financial markets have made manual trading inefficient and unreliable in many scenarios. Traders are required to analyze multiple

indicators, monitor various assets, and respond instantly to market movements. Such demands exceed human cognitive and physical limitations.

Automation addresses these challenges by enabling systems to process data continuously and execute decisions without human intervention. Automated trading systems can monitor markets 24/7, analyze multiple assets simultaneously, and respond instantly to predefined conditions.

Data processing is a critical component of automation. Raw market data must be cleaned, transformed, and analyzed before meaningful conclusions can be drawn. Techniques such as data preprocessing, indicator calculation, and performance evaluation form the foundation of algorithmic decision-making.

Backtesting plays an important role in validating automated strategies. By simulating trades on historical data, traders can evaluate the effectiveness and risk of strategies before applying them in live markets. This reduces uncertainty and promotes informed decision-making.

The proposed platform integrates automation and data processing to provide users with a structured approach to trading analysis. It emphasizes clarity, reliability, and systematic evaluation of strategies.

1.7 Educational, Practical, And Industry Significance Of The Project

Algorithmic trading has emerged as one of the most important applications of information technology in the financial domain. With the increasing adoption of automated systems in stock markets and investment firms, there is a growing demand for professionals who possess both technical and financial knowledge. From an academic perspective, studying algorithmic trading provides students with an opportunity to understand how computing, data analysis, and decision-making techniques are applied in real-world financial environments.

This project holds significant educational value as it allows students to move beyond theoretical concepts and gain hands-on experience with practical tools and technologies. By working on the Algorithmic Trading Web Platform, students are exposed to real market data, programming logic, and analytical workflows. Concepts such as data preprocessing, indicator calculation, backtesting, and performance evaluation help reinforce learning outcomes from

subjects like data structures, databases, statistics, and software engineering. This integration of multiple disciplines makes the project academically rich and relevant.

From a practical standpoint, the project enables learners to understand the complete lifecycle of an algorithmic trading system. Instead of focusing only on trading strategies, the platform highlights the importance of data handling, system design, error management, and result interpretation. Students learn how raw market data is transformed into meaningful insights and how algorithmic rules are systematically applied to generate trading signals. This practical exposure helps build problem-solving skills and improves the ability to design structured and efficient software systems.

The project also encourages responsible and risk-aware trading practices by emphasizing backtesting and strategy validation. Beginners often enter financial markets without adequate testing or understanding of risk, leading to financial losses. By simulating strategies on historical data, the platform helps users evaluate the strengths and weaknesses of their strategies before applying them in real-world scenarios. This approach promotes analytical thinking and disciplined decision-making, which are essential skills in both academic and professional environments.

In terms of industry relevance, algorithmic trading is widely used by banks, hedge funds, asset management companies, and fintech startups. The financial industry increasingly relies on automation to handle large-scale transactions, reduce operational costs, and improve execution efficiency. Familiarity with algorithmic trading systems, data-driven decision-making, and analytical platforms provides a strong foundation for careers in fields such as financial technology, quantitative analysis, data science, and software development.

Moreover, web-based trading platforms reflect current industry trends where accessibility, scalability, and user experience are prioritized. The use of web technologies allows systems to be easily deployed, updated, and accessed from different environments. By developing a web-based algorithmic trading platform, this project aligns closely with real-world practices and industry expectations.

The modular and extensible nature of the project further enhances its long-term value. The platform can be extended to include advanced features such as machine learning models, risk management frameworks, multi-asset support, and broker integration. This makes the project

suitable not only as a final-year academic submission but also as a foundation for future research, internships, or entrepreneurial initiatives in the fintech domain.

In conclusion, the educational, practical, and industry significance of the Algorithmic Trading Web Platform lies in its ability to combine academic learning with real-world applicability. The project equips students with essential technical skills, promotes analytical and ethical decision-making, and provides exposure to industry-relevant technologies. As a result, it serves as a meaningful and future-oriented academic endeavor that bridges the gap between classroom learning and professional practice.

1.8 Objectives And Organization Of The Report

1.8.1 Introduction To Project Objectives

Every academic project is driven by a clear set of objectives that define its purpose, direction, and expected outcomes. Project objectives act as a guiding framework that helps in planning, designing, implementing, and evaluating the overall system. In the context of a technical project, objectives ensure that the development process remains focused on solving the identified problem while meeting academic and practical requirements.

The Algorithmic Trading Web Platform is designed with well-defined objectives that address both technical implementation and learning outcomes. These objectives provide clarity regarding what the project aims to achieve and how success can be measured. They also help in aligning the project work with real-world applications and industry practices, making the project relevant beyond academic evaluation.

In modern software development projects, especially those related to financial systems, objectives must consider multiple dimensions such as functionality, performance, usability, scalability, and reliability. Algorithmic trading systems operate in a dynamic and data-intensive environment, where accuracy and efficiency are critical. Therefore, defining clear objectives is essential to ensure that the system delivers meaningful results and adheres to expected standards.

From an academic perspective, project objectives also help evaluators understand the intent and scope of the work. They serve as a reference point against which the quality, completeness, and effectiveness of the project can be assessed. This section introduces the

importance of project objectives and sets the foundation for understanding the specific goals of the Algorithmic Trading Web Platform.

1.8.2 General Objectives Of The Project

The general objectives describe the broad goals that the project aims to accomplish. These objectives focus on the overall purpose of the system rather than specific technical details. For the Algorithmic Trading Web Platform, the general objectives are centered around developing a functional, user-friendly, and educational trading analysis system.

One of the primary general objectives of this project is to design and develop a web-based platform that demonstrates the working principles of algorithmic trading. The platform aims to provide users with an understanding of how automated trading systems analyze market data and generate trading decisions based on predefined rules.

Another key objective is to simplify complex trading concepts and present them in an accessible manner. Financial markets and algorithmic trading can be difficult for beginners to understand. This project seeks to reduce that complexity by offering a structured and interactive interface that allows users to explore trading strategies without requiring advanced financial or programming expertise.

The project also aims to promote data-driven decision-making by encouraging users to rely on analytical results rather than intuition or emotion. By providing tools for backtesting and visualization, the platform helps users understand the importance of evidence-based trading strategies.

Additionally, a general objective of the project is to integrate theoretical knowledge with practical implementation. Concepts learned in subjects such as programming, data structures, databases, and statistics are applied in a real-world context. This integration enhances the educational value of the project and supports experiential learning.

1.8.3 Technical Objectives

The technical objectives focus on the implementation aspects of the project and define what the system should technically achieve. These objectives ensure that the project meets expected standards of software design, functionality, and performance.

One of the main technical objectives is to implement a reliable mechanism for fetching real-time and historical market data using APIs. Accurate data retrieval is essential for performing meaningful analysis and strategy evaluation. The system must be capable of handling data in structured formats and managing potential issues such as missing values or API limitations.

Another important technical objective is to implement rule-based algorithmic trading strategies. These strategies are defined using mathematical and logical conditions that determine when to enter or exit trades. The project focuses on implementing basic yet effective strategies to demonstrate the core principles of algorithmic trading.

The development of a backtesting module is also a critical technical objective. Backtesting allows the system to simulate trading strategies on historical data and evaluate their performance. This involves accurate calculation of metrics such as profit, loss, returns, and drawdowns.

The project also aims to achieve effective data visualization through charts and tables. Visual representation of price movements, indicators, and performance metrics helps users interpret results more easily. The technical objective here is to present complex data in a clear and intuitive format.

Finally, the system is designed to be modular and extensible. This technical objective ensures that new strategies, data sources, or features can be added in the future without major changes to the existing system architecture.

1.8.4 Functional Objectives

Functional objectives define what the system is expected to do from a user's perspective. These objectives focus on user interaction, system behavior, and overall usability.

A key functional objective of the Algorithmic Trading Web Platform is to provide an intuitive and easy-to-use interface. Users should be able to navigate the platform, select trading strategies, view market data, and analyze results without facing unnecessary complexity.

Another functional objective is to allow users to select financial instruments and time periods for analysis. This flexibility enables users to experiment with different market conditions and understand how strategies perform over various time frames.

The platform is also designed to function as an analytical tool rather than a live trading system. A functional objective of the project is to simulate trading behavior safely through backtesting, ensuring that users can experiment without financial risk.

Providing clear and structured output is another functional objective. The system should present results in an organized manner, including performance summaries, charts, and data tables. This helps users draw meaningful conclusions from the analysis.

Additionally, the platform aims to function as an educational resource. By displaying strategy logic and performance metrics, the system helps users learn how algorithmic trading decisions are formed and evaluated.

1.8.5 Non-Functional Objectives

Non-functional objectives define the quality attributes and constraints of the system rather than specific features. These objectives are critical for ensuring that the system performs reliably and meets user expectations.

One important non-functional objective is system performance. The platform should process data and generate results within a reasonable time frame, even when working with large datasets. Efficient data handling and computation are essential to achieve this objective.

Reliability is another key non-functional objective. The system should produce consistent and accurate results when provided with the same input data. Errors related to data handling or strategy execution must be minimized.

Scalability is also considered as a non-functional objective. Although the current project is designed for academic use, the system architecture should allow future expansion, such as supporting additional strategies or larger datasets.

Usability and maintainability are equally important non-functional objectives. The platform should be easy to use for beginners and easy to maintain or update by developers. Clear code structure and modular design support these objectives.

Security is another non-functional objective, particularly in handling API keys and configuration data. Sensitive information must be managed carefully to prevent unauthorized access.

1.8.6 Learning And Skill Development Objectives

One of the important objectives of an academic project is to enhance the learning experience of students by providing practical exposure to real-world problems. The Algorithmic Trading Web Platform is designed with strong learning and skill development objectives that go beyond theoretical understanding.

A major learning objective of this project is to strengthen programming skills, particularly in Python and related libraries used for data analysis and visualization. By implementing trading strategies, handling APIs, and processing financial data, students gain hands-on experience in writing structured, efficient, and readable code.

Another learning objective is to develop analytical thinking and problem-solving abilities. Algorithmic trading requires breaking down complex problems into smaller logical components, such as data preprocessing, indicator calculation, and decision-making rules. This structured approach helps students improve their ability to analyze problems systematically and design effective solutions.

The project also aims to enhance understanding of financial market concepts. Students learn how market data behaves, how trends are identified, and how trading decisions impact performance. This interdisciplinary learning helps bridge the gap between computer science concepts and financial applications.

Additionally, the project helps in developing skills related to data interpretation and visualization. Presenting large volumes of numerical data in graphical and tabular formats enables students to communicate insights clearly and effectively. These skills are valuable in both academic and professional environments.

Overall, the learning and skill development objectives of this project contribute to holistic growth by combining technical expertise, analytical ability, and domain knowledge.

1.8.7 Business And Practical Objectives

The business and practical objectives focus on ensuring that the project reflects real-world applicability and aligns with industry practices. Algorithmic trading is widely used in the financial sector to improve efficiency, reduce costs, and enhance decision-making accuracy. This project aims to simulate these practical aspects in an academic setting.

One of the key practical objectives is to demonstrate how automated systems can assist traders in making informed decisions. By analyzing historical data and strategy performance, the platform provides insights that would be difficult to obtain through manual analysis alone.

Another objective is to highlight the importance of cost-effective and scalable solutions. Many commercial trading platforms are expensive and complex. This project aims to show how open-source tools and web technologies can be used to build functional trading analysis systems at a low cost.

The project also emphasizes practical usability. The platform is designed to be user-friendly, ensuring that even beginners can interact with it effectively. This practical focus enhances the real-world relevance of the project.

From a business perspective, the project demonstrates how data-driven platforms can create value by transforming raw data into actionable insights. Such platforms are essential in modern financial services, fintech startups, and analytics-driven organizations.

1.8.8 Short-Term Objectives

Short-term objectives define the immediate goals that the project aims to achieve during its development phase. These objectives ensure that the project progresses in a structured and manageable manner.

One short-term objective is to understand the basic concepts of algorithmic trading and market data analysis. This includes studying trading strategies, indicators, and performance metrics relevant to automated trading systems.

Another short-term objective is to design the system architecture and user interface. This involves deciding how different modules such as data fetching, strategy execution, backtesting, and visualization will interact with each other.

Implementing core functionalities is also a key short-term objective. These include fetching market data, applying trading strategies, performing backtesting, and displaying results. Achieving these milestones ensures that the project remains on track.

Testing and validation form another important short-term objective. The system must be tested with different datasets and scenarios to ensure correctness and reliability.

These short-term objectives help break down the project into achievable tasks, making the development process systematic and efficient.

1.8.9 Long-Term Objectives

Long-term objectives define the future potential and growth of the project beyond its initial implementation. These objectives ensure that the project remains relevant and adaptable to changing requirements.

One long-term objective is to extend the platform with advanced trading strategies and analytical models. This may include integrating machine learning techniques for predictive analysis and pattern recognition.

Another objective is to enhance scalability by supporting multiple assets, markets, and larger datasets. This would allow the platform to handle more complex analysis and broader use cases.

The project also has the long-term objective of integrating live trading features using broker APIs, subject to regulatory and safety considerations. This would transform the platform from an analytical tool into a complete trading system.

Additionally, the platform can be developed further as a research or entrepreneurial initiative in the fintech domain. Its modular design makes it suitable for future enhancements and collaborations.

1.8.10 Scope of the Project

The scope of the project defines its boundaries and limitations, ensuring clarity regarding what the system includes and excludes. Clearly defining the scope helps manage expectations and maintain focus during development.

The scope of the Algorithmic Trading Web Platform includes market data analysis, implementation of rule-based trading strategies, backtesting, and visualization. The platform is designed primarily for educational and analytical purposes.

Live trading execution and real-money transactions are outside the immediate scope of this project. This limitation ensures safety and compliance while allowing users to experiment freely with strategies.

The project focuses on selected financial instruments and basic strategies rather than covering all possible market scenarios. This controlled scope helps maintain simplicity and clarity.

By defining a clear scope, the project ensures effective utilization of time and resources while delivering meaningful outcomes aligned with academic objectives.

Chapter 2

Review of Past Work and Problem Formulation

2.1 Overview

Algorithmic trading has emerged as a significant advancement in the financial markets, enabling the execution of trades based on predefined rules and computational logic rather than manual decision-making. With the rapid growth of digital trading platforms and the increasing availability of market data, automated trading systems have gained widespread adoption among institutional and retail investors alike. These systems aim to improve trading efficiency by minimizing human intervention, reducing emotional bias, and enabling faster execution of trades.

Past research in algorithmic trading primarily focuses on the use of technical indicators, statistical models, and rule-based strategies to analyze market behavior and generate trading signals. Many existing studies highlight the advantages of algorithm-driven trading, including improved consistency, enhanced accuracy, and the ability to evaluate strategies using historical data through backtesting. However, several traditional systems lack user-friendly interfaces and do not provide adequate visualization or comparative analysis of trading strategies.

This chapter presents a review of past work related to algorithmic trading systems and examines the limitations of existing approaches. It also provides the foundation for identifying the research gap and formulating the problem addressed in this project. By analyzing previous methodologies and tools, this chapter establishes the need for a structured, scalable, and accessible algorithmic trading web platform capable of performing strategy evaluation and result visualization.

2.2 Types of Algorithmic Trading Strategies

Algorithmic trading systems can be broadly classified based on the logic and methodologies used to generate trading decisions. Over the years, various types of trading strategies have been proposed and implemented, each with its own advantages and limitations. The major categories of algorithmic trading strategies relevant to this project are discussed below.

❖ Rule-Based Trading Strategies

Rule-based trading strategies rely on predefined logical rules derived from technical indicators and market patterns. These strategies generate buy or sell signals when specific conditions are met, such as price crossing a moving average or momentum reaching a threshold. Due to their simplicity and interpretability, rule-based strategies are widely used for backtesting and educational purposes and form the core approach of this project.

❖ **Indicator-Based Trading Strategies**

Indicator-based strategies utilize technical indicators such as Moving Averages, Relative Strength Index (RSI), and Moving Average Convergence Divergence (MACD) to analyze price movements and market trends. These indicators help identify overbought or oversold conditions and potential trend reversals. Such strategies are computationally efficient and suitable for automated execution in algorithmic trading systems.

❖ **Statistical and Quantitative Strategies**

Statistical trading strategies are based on mathematical and probabilistic models that analyze historical price data to identify patterns or anomalies. These strategies often involve concepts such as mean reversion, correlation analysis, and statistical arbitrage. While effective in certain market conditions, they require robust data preprocessing and parameter tuning.

❖ **Machine Learning-Based Trading Strategies**

Machine learning-based strategies employ algorithms such as decision trees, support vector machines, and neural networks to predict market trends and generate trading signals. These strategies learn from historical data and adapt to changing market conditions. Although they offer high potential accuracy, they are complex to implement and require large datasets and computational resources.

❖ **Hybrid Trading Strategies**

Hybrid strategies combine multiple approaches, such as technical indicators and machine learning models, to improve prediction accuracy and reduce risk. By integrating different analytical techniques, hybrid systems aim to overcome the limitations of individual strategies. These approaches are often used in advanced trading platforms and research-oriented applications.

2.3 Review of Existing Algorithmic Trading Systems

Over the past decade, algorithmic trading systems have gained significant attention in both academic research and practical financial markets. Existing systems primarily aim to automate trading decisions by analyzing historical and real-time market data using predefined rules, statistical models, or advanced computational techniques. These systems are widely used by institutional investors, hedge funds, and professional traders to improve execution speed and trading efficiency.

Many traditional algorithmic trading platforms focus on rule-based and indicator-driven strategies, where buy and sell signals are generated using technical indicators such as moving averages, RSI, and MACD. While these systems are relatively easy to implement and interpret, they often require manual parameter tuning and lack flexibility in adapting to changing market conditions. Additionally, several existing systems emphasize trade execution rather than strategy evaluation, offering limited backtesting capabilities.

Advanced algorithmic trading systems incorporate quantitative and machine learning techniques to predict price movements and optimize trading decisions. These systems leverage large datasets and complex models to identify hidden patterns in financial data. However, such platforms are often computationally intensive, expensive, and difficult to understand, making them less accessible to individual traders and students.

Another limitation observed in existing systems is the lack of effective visualization and user interaction. Many platforms provide numerical performance metrics without intuitive graphical representations, which makes result interpretation difficult. Furthermore, some systems do not allow easy comparison of multiple strategies, thereby restricting comprehensive performance analysis.

In summary, although existing algorithmic trading systems offer automation and speed, they suffer from limitations such as complexity, high cost, insufficient visualization, and restricted accessibility. These challenges highlight the need for a simplified, user-friendly, and extensible algorithmic trading platform that emphasizes backtesting, performance analysis, and clear visualization—motivating the development of the proposed system.

2.4 Limitations of Existing Systems

Area	Limitations of Existing Systems	Proposed System (This Project)
Focus on Strategy Evaluation	Existing systems primarily emphasize automation and execution speed, with limited focus on evaluating and validating trading strategies before deployment.	Integrates comprehensive strategy evaluation and validation before deployment to reduce financial risk.
Backtesting Capability	Many platforms provide limited or no comprehensive backtesting using historical market data.	Provides an integrated backtesting framework using historical market data under different market conditions.
Performance Visualization	Backtesting results are often presented only in numerical form, making analysis and interpretation difficult.	Offers graphical visualization of performance metrics for easier analysis and better decision-making.
Complexity & Accessibility	Machine learning-based platforms require large datasets, high computational resources, and expert-level knowledge.	Designed as a simplified, user-friendly platform suitable for students, researchers, and retail traders.
Cost & Availability	Professional algorithmic trading platforms are often expensive, proprietary, and closed-source.	Provides a cost-effective, extensible, and transparent platform for experimentation and learning.
Strategy Comparison & Customization	Existing systems offer limited support for comparing multiple strategies and customizing trading approaches.	Enables easy comparison of multiple trading strategies with flexible customization options.

User Interface & Usability	Many platforms have complex and non-intuitive user interfaces, reducing usability and adoption.	Features a modern, intuitive, and user-friendly interface to improve accessibility and user experience.
Research Gap	Existing systems fail to provide an integrated environment combining backtesting, strategy evaluation, visualization, and ease of use.	Develops a unified algorithmic trading platform that integrates backtesting, strategy evaluation, graphical visualization, and an intuitive interface.

2.5 Problem Formulation

The rapid growth of financial markets and the increasing availability of market data have made algorithmic trading an essential tool for modern trading practices. However, as identified in the review of existing systems, current trading platforms suffer from limitations such as high complexity, lack of accessibility, insufficient backtesting capabilities, and inadequate visualization of trading performance. These challenges make it difficult for students, researchers, and individual traders to effectively design, test, and evaluate trading strategies.

The primary problem addressed in this project is the absence of a simplified and user-friendly algorithmic trading system that enables users to validate trading strategies before actual deployment. There is a need for a platform that can automatically generate trading signals based on predefined rules, execute trades in a simulated environment using historical data, and provide clear performance evaluation through graphical and tabular outputs.

Therefore, the problem can be formulated as the design and development of a web-based algorithmic trading platform that integrates strategy selection, historical data analysis, backtesting, and result visualization within a single framework. The system should allow users to test multiple strategies, analyze profitability and risk, and understand trading behavior without requiring advanced technical expertise or high computational resources. By addressing this problem, the proposed system aims to bridge the gap between complex professional trading platforms and basic manual trading methods, providing an effective solution for learning, analysis, and decision support in algorithmic trading.

2.6 Summary of the Chapter

This chapter presented a comprehensive review of past work related to algorithmic trading systems and examined the various approaches used in automated trading. An overview of algorithmic trading and its significance in modern financial markets was discussed, followed by a classification of different types of algorithmic trading strategies. Existing algorithmic trading systems were reviewed to understand their methodologies, strengths, and limitations.

The chapter further identified key research gaps and limitations in current systems, such as lack of user-friendly interfaces, insufficient backtesting and visualization features, and high complexity of advanced trading platforms. Based on these observations, the problem addressed by this project was clearly formulated, emphasizing the need for a simplified, accessible, and extensible algorithmic trading platform.

Overall, this chapter establishes a strong foundation for the proposed system and provides a clear rationale for the design and methodology discussed in the subsequent chapters.

Chapter 3

System Analysis & Requirement

3.1 Introduction

System analysis is a crucial phase in the development of any software project, as it helps in understanding the problem domain, identifying user needs, and defining the functional boundaries of the system. In the context of the Algorithmic Trading Web Platform, system analysis ensures that the proposed solution effectively addresses the limitations of traditional trading systems while fulfilling academic and practical objectives.

This chapter presents a detailed analysis of the system, including problem identification, study of the existing system, limitations of current approaches, and the design of the proposed system. It also outlines the functional and non-functional requirements necessary for the successful implementation of the platform.

3.2 Problem Definition

Traditional trading methods rely heavily on manual decision-making, emotional judgment, and time-consuming analysis of market data. Traders often face difficulties in analyzing large volumes of financial data, identifying patterns, and executing trades at the right moment. Human errors, emotional bias, and delayed reactions can lead to inconsistent trading performance and financial losses.

Additionally, many existing trading platforms are complex, expensive, and require advanced financial knowledge, making them inaccessible to beginners and small investors. The lack of transparency in strategy performance and limited backtesting capabilities further complicate informed decision-making.

The problem addressed by this project is the need for an automated, user-friendly, and analytical trading platform that can assist users in evaluating trading strategies using historical data, reducing emotional bias, and improving decision accuracy.

3.3 Existing System

In the existing system, most retail traders depend on manual trading techniques or basic charting tools. These systems generally require users to continuously monitor market movements, interpret charts, and make decisions based on intuition or limited indicators.

Professional trading platforms do offer advanced features, but they often come with high subscription costs and steep learning curves. Many platforms also lack customization options for strategy testing and performance evaluation.

Furthermore, manual trading systems are inefficient when dealing with large datasets and fast-moving markets. The absence of automated backtesting and real-time analysis restricts traders from evaluating strategies thoroughly before applying them in real markets.

3.4 Proposed System

The proposed system is an **Algorithmic Trading Web Platform** that automates market data analysis and strategy evaluation using computational techniques. The platform is designed to assist users in understanding market behavior and evaluating trading strategies through backtesting and visualization.

The system allows users to analyze historical market data, apply predefined trading strategies, and observe performance metrics such as profit, loss, and trade accuracy. By automating these processes, the platform minimizes emotional influence and improves analytical accuracy.

The proposed system emphasizes simplicity, accessibility, and scalability, making it suitable for students, beginners, and researchers interested in algorithmic trading.

3.5 Objectives of the Proposed System

The objectives of the proposed system include:

- Automating trading strategy evaluation
- Reducing human errors and emotional bias

- Providing an interactive and user-friendly interface
- Enabling backtesting using historical data
- Improving data visualization and analysis
- Offering a cost-effective alternative to commercial platforms

These objectives ensure that the system delivers both academic and practical value.

3.6 Functional Requirements

• User Registration

The system should allow new users to create an account by providing their username, email address, and password. It should prevent duplicate registrations using the same email address.

• User Login

The system should authenticate registered users using their email and password and provide secure access to the platform through JWT authentication.

• User Profile Management

The system should allow users to view and manage their personal profile information after logging into the application.

• Stock Search

The system should enable users to search for stocks using their company name or stock symbol to quickly access market information.

• Watchlist Management

The system should allow users to create and manage a personalized watchlist by adding or removing their preferred stocks.

• Real-Time Market Data

The system should fetch and display real-time market prices, including the latest stock price, price change, and percentage change.

- **Historical Market Data**

The system should retrieve historical stock data for different time periods, allowing users to analyze past market trends.

- **Interactive Candlestick Charts**

The system should display stock price movements using interactive candlestick charts for better visualization and technical analysis.

- **Multiple Timeframe Support**

The system should allow users to switch between different chart intervals such as 1 minute, 5 minutes, 15 minutes, 1 hour, and 1 day according to their trading requirements.

- **Technical Indicator Support**

The system should support various technical indicators such as RSI, EMA, MACD, Golden Cross, and SuperTrend for market analysis.

- **Strategy Selection**

The system should allow users to select from multiple predefined trading strategies before executing backtesting or paper trading.

- **Strategy Configuration**

The system should allow users to customize strategy parameters such as moving average periods and RSI thresholds based on their trading preferences.

- **Historical Backtesting**

The system should execute trading strategies on historical market data to evaluate their performance before applying them to live markets.

- **Backtesting Results**

The system should display detailed backtesting reports including total return, number of trades, win rate, loss rate, drawdown, and overall profitability.

- **Paper Trading**

The system should simulate real-time trading using virtual money, enabling users to test strategies without financial risk.

- **Paper Trading Control**

The system should provide options to start, stop, and monitor paper trading sessions with live market data.

- **Live Trading**

The system should support live order execution by integrating with a broker's trading API after successful authentication.

- **Trading Signal Generation**

The system should automatically generate Buy, Sell, or Hold signals based on the selected trading strategy and current market conditions.

- **Position Management**

The system should maintain records of all active and closed positions, including entry price, exit price, quantity, and profit or loss.

- **Trade History**

The system should maintain a complete history of all executed trades along with their execution time, price, quantity, and order type.

- **Portfolio Management**

The system should display the user's portfolio, including current holdings, investment value, available balance, and overall portfolio performance.

- **Profit and Loss Calculation**

The system should calculate and display real-time as well as cumulative profit and loss for each trade and the entire portfolio.

- **Order Management**

The system should allow users to place, modify, cancel, and monitor trading orders during live trading sessions.

- **Broker Integration**

The system should securely connect with the Fyers API to authenticate users, retrieve market data, and execute live orders.

- **Dashboard**

The system should provide a centralized dashboard displaying trading statistics, portfolio summary, recent trades, active positions, and market information.

- **Performance Analytics**

The system should generate analytical reports and graphical representations of trading performance, equity growth, and portfolio returns.

- **Risk Management**

The system should implement risk management features such as stop-loss, take-profit, position sizing, and maximum investment limits to reduce trading risk.

- **Notifications**

The system should notify users about important events such as successful order execution, strategy status changes, trading signals, and system alerts.

- **Secure Data Storage**

The system should securely store user information, trading history, portfolio details, strategy configurations, and system logs in the database.

- **API Security**

The system should secure all protected APIs using JWT authentication to ensure that only authorized users can access sensitive resources.

- **Activity Logging**

The system should maintain logs of trading activities, API requests, and system events to assist in monitoring, debugging, and future analysis.

These requirements ensure smooth and meaningful system functionality.

3.7 Feasibility Analysis

Feasibility analysis evaluates whether the project can be successfully implemented within given constraints.

3.7.1 Technical Feasibility

The project is technically feasible due to the availability of open-source tools, libraries, and APIs that support data analysis and web development.

3.7.2 Economic Feasibility

The project is cost-effective as it relies primarily on free and open-source technologies.

3.7.3 Operational Feasibility

The system is easy to operate and requires minimal training, making it suitable for academic and.

3.8 Summary

This chapter presented a comprehensive system analysis of the Algorithmic Trading Web Platform. It discussed the existing system, identified its limitations, and proposed an improved solution with clearly defined objectives and requirements. The analysis establishes a strong foundation for system design and implementation, which are discussed in subsequent chapters.

Chapter 4

System design

4.1 Introduction

System design is a critical phase in the software development life cycle that translates system requirements into a structured and implementable blueprint. It defines how different components of the system interact with each other to achieve the desired functionality. A well-designed system improves reliability, scalability, and maintainability.

This chapter presents the detailed system design of the Algorithmic Trading Web Platform. It includes system architecture, data flow diagrams, use case diagrams, module design, and interface design. These design elements provide a clear understanding of the internal structure and working of the proposed system.

4.2 System Architecture

The system architecture defines the overall structure of the platform and the interaction between its major components. The proposed system follows a modular architecture to ensure flexibility and scalability.

The main components of the system architecture include:

- User Interface
- Application Logic
- Data Processing Module
- Strategy Engine
- Database System
- External Data Sources (APIs)

The user interacts with the system through a web-based interface. The application logic handles user requests and coordinates with the data processing and strategy modules. External APIs provide market data, which is processed and stored in the database. The strategy engine applies trading rules and generates analytical outputs.

This layered architecture ensures separation of concerns and simplifies future enhancements.

4.3 Architectural Design Description

The system adopts a client–server architecture, where the front-end interacts with the backend processing modules. The backend performs data analysis, strategy evaluation, and result computation.

Each module operates independently but communicates through well-defined interfaces. This design approach enhances maintainability and allows individual components to be updated without affecting the entire system.

The architecture also supports scalability by allowing additional modules such as advanced analytics or machine learning to be integrated in the future.

4.4 Data Flow Diagram (DFD)



Fig 01 DFD level

Data Flow Diagrams illustrate how data moves through the system and how it is processed at each stage.

❖ **Level 0 DFD (Context Diagram)**

The Level 0 DFD represents the entire system as a single process. The user provides input such as asset selection and strategy parameters. The system processes the input and produces outputs such as performance metrics and visualizations.

External entities include:

- User
- Market Data Provider

The system acts as an intermediary that processes input data and generates meaningful results.

❖ **Level 1 DFD**

The Level 1 DFD breaks the system into major processes:

- Data Collection
- Data Processing
- Strategy Execution
- Backtesting, Paper Trading, Live Trading. Recommendation System
- Result Visualization

Data flows sequentially through these processes, ensuring structured and efficient information handling.

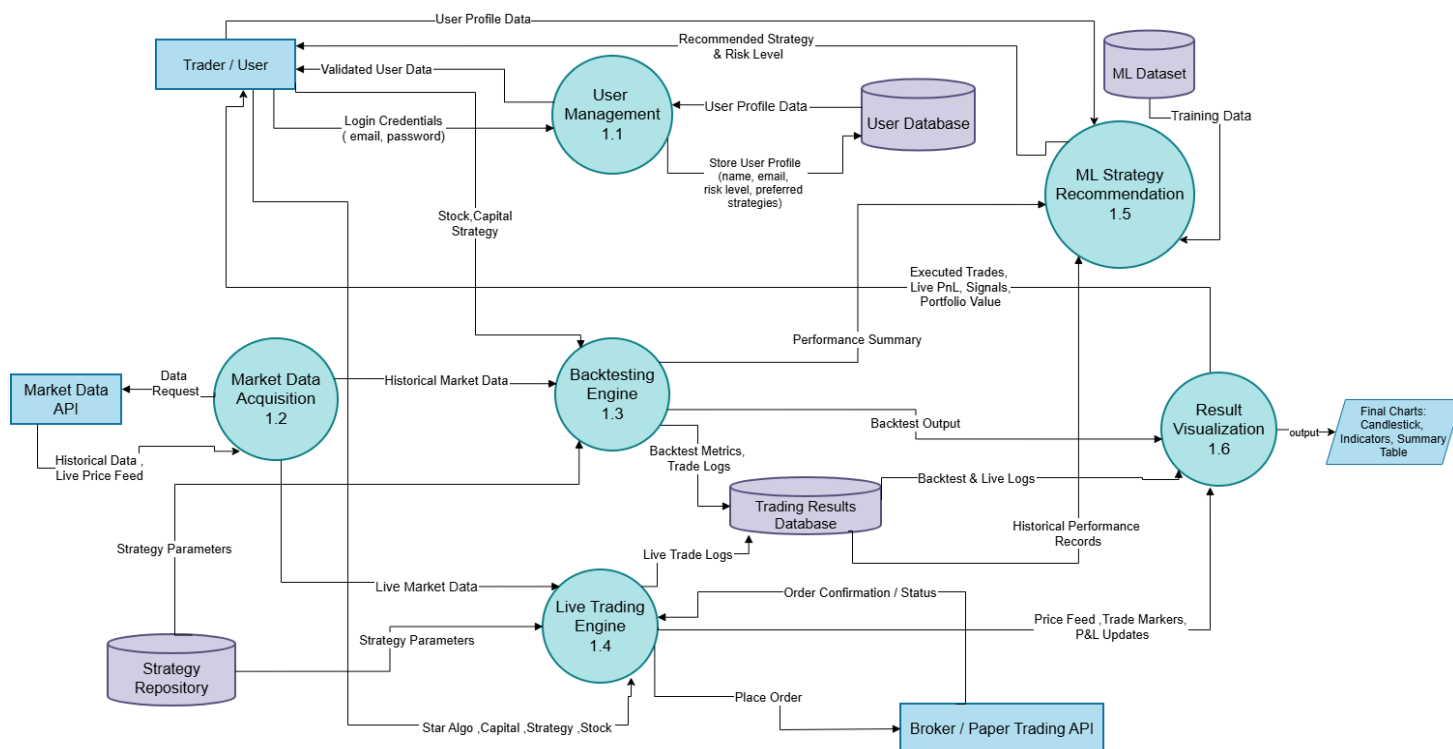


Fig 02 DFD level 1

❖ Level 2 DFD

The Level 2 Data Flow Diagram (DFD) provides a more detailed view of the system's internal processes. It breaks down the major modules from Level 1 into sub-processes, illustrating how data is handled, transformed, and transmitted between components. This detailed representation helps in understanding the finer operational workflow of the Algorithmic Trading Web Platform and ensures clarity in system implementation.

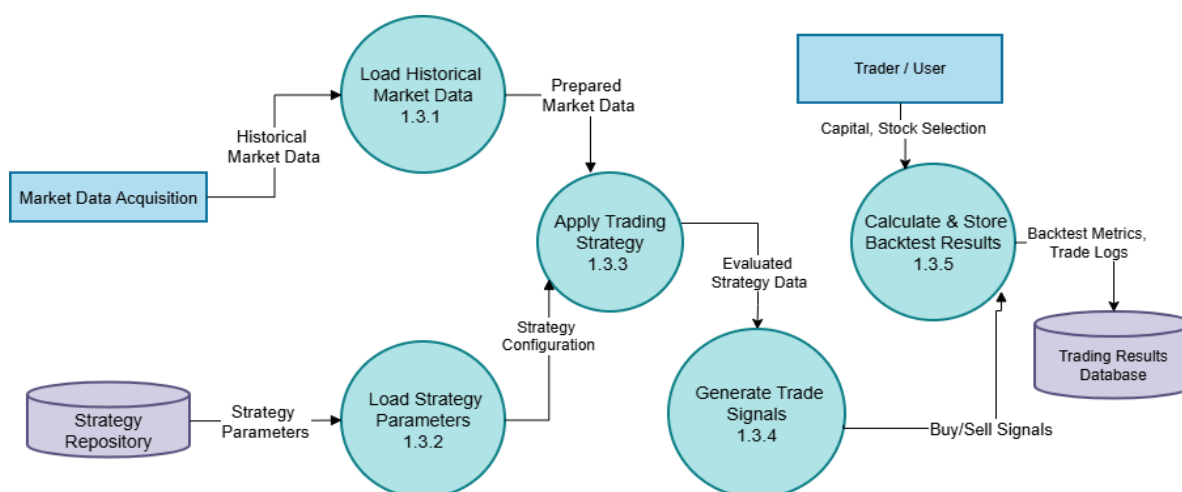


Fig 03: DFD level 2 Backtest Module

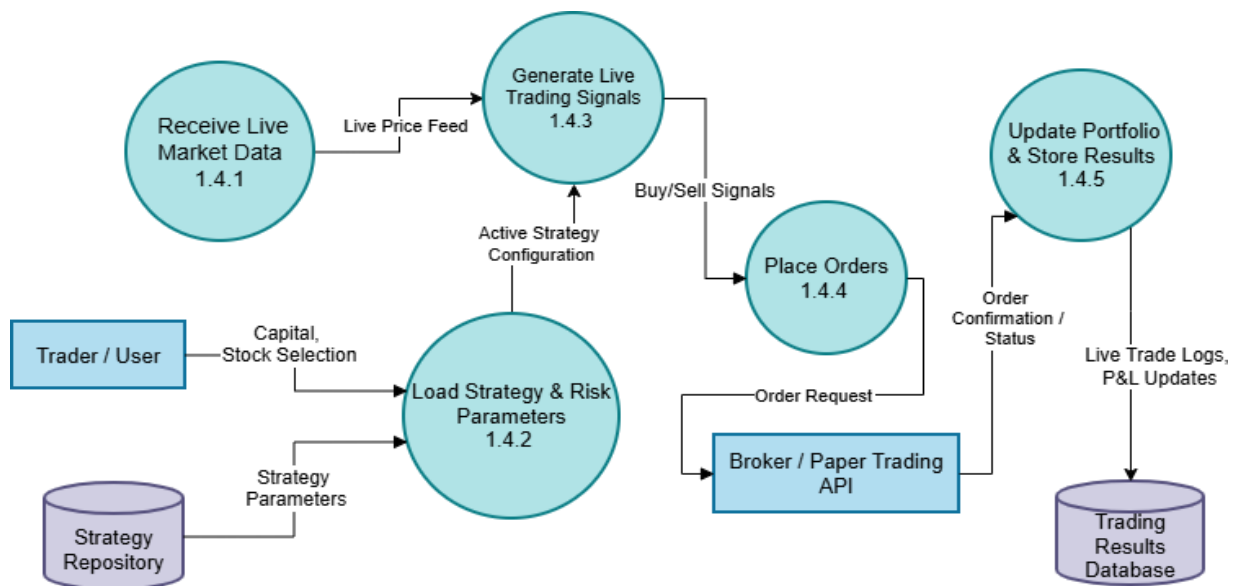


Fig 04 : DFD level 2 Live Trading Module

4.5 ER Diagram

The Entity-Relationship (ER) diagram represents the logical structure of the database used in the Algorithmic Trading Web Platform. It illustrates the entities, their attributes, and the relationships between them, providing a clear view of how data is organized and interconnected. The ER diagram serves as a blueprint for database design, ensuring data integrity, efficient storage, and seamless retrieval for processing and analysis.

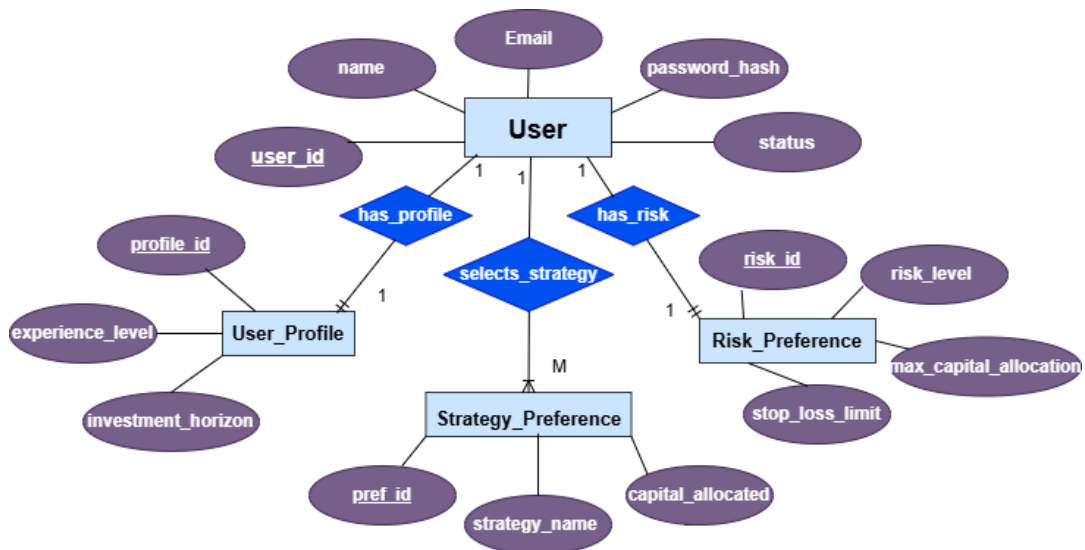


Fig 05 : ER diagram

Chapter 5

Algorithm and Trading Strategies

5.1 Introduction

Algorithms form the core of the Algorithmic Trading Web Platform. In algorithmic trading, predefined rules and computational logic are applied to market data to make trading decisions automatically. This chapter presents the algorithms and strategies implemented in the platform, their operational flow, and the logic behind decision-making.

Trading strategies aim to identify profitable opportunities while minimizing risks. By automating these strategies, the system can analyze large datasets, apply rules consistently, and provide reliable backtesting results.

5.2 Role of Algorithms in Algorithmic Trading

Algorithms in trading automate the decision-making process by applying mathematical and statistical rules to financial data. They enable:

- Fast and accurate analysis of market data
- Elimination of emotional bias in trading decisions
- Consistent application of trading strategies
- Backtesting of strategies against historical data
- Efficient risk management through rule-based logic

Without well-designed algorithms, trading decisions would remain manual, slower, and prone to human error.

5.3 Types of Trading Strategies

The platform implements rule-based strategies that can be categorized as follows:

❖ Trend-Following Strategies

These strategies identify market trends and generate buy/sell signals accordingly. Examples include:

- Moving Average Crossover
- Momentum-based strategies

Logic: Buy when the short-term moving average crosses above the long-term moving average; sell when it crosses below.

❖ Mean Reversion Strategies

These strategies assume that prices revert to their historical average. When prices deviate significantly, a trade is executed expecting a return to the mean.

Logic: If the current price is above the historical average by a certain threshold, sell; if below, buy.

❖ Arbitrage Strategies

These exploit price differences across markets or instruments. Although not implemented in full, the platform demonstrates the principle through simplified simulations.

Logic: Identify price discrepancies and simulate simultaneous buying and selling to profit from differences.

5.4 Algorithm Design and Pseudocode

The platform uses structured pseudocode to implement strategies:

Example: Moving Average Crossover Strategy

1. Fetch historical price data for the selected asset
2. Calculate short-term and long-term moving averages
3. For each trading period:
 - a. If short-term MA $>$ long-term MA:
Generate BUY signal
 - b. Else if short-term MA $<$ long-term MA:
Generate SELL signal
4. Record trades and calculate profit/loss
5. Display results in charts and tables

This pseudocode ensures clarity and provides a foundation for implementation in Python.

5.5 Strategy Execution Flow

The execution flow of trading strategies follows these steps:

1. **Data Input:** Historical or real-time market data is fetched.
2. **Preprocessing:** Data is cleaned, normalized, and structured.
3. **Indicator Calculation:** Relevant indicators (moving averages, RSI, etc.) are computed.
4. **Rule Evaluation:** Strategy logic is applied to generate trading signals.
5. **Trade Simulation:** Trades are simulated and recorded for backtesting.
6. **Performance Metrics:** Returns, profit/loss, and other key metrics are calculated.
7. **Result Visualization:** Outputs are displayed using charts and tables.

This flow ensures systematic processing and consistent evaluation of trading strategies.

5.6 Backtesting Methodology

Backtesting evaluates the performance of a strategy using historical data. Key steps include:

- Selecting a historical time frame
- Applying strategy rules to historical prices
- Recording trades and computing performance metrics
- Analyzing results for profitability and risk

Backtesting helps identify strengths and weaknesses of strategies before deploying them in live markets. The platform includes visualizations for cumulative returns, trade frequency, and drawdowns.

5.7 Live/Paper Trading Methodology

Live (Paper) Trading simulates real-time trading by executing virtual trades using live market data without involving actual financial transactions. The methodology consists of the following steps:

- **Fetching live market data** at regular intervals from the market data provider.
- **Applying the selected trading strategy** to the incoming OHLCV (Open, High, Low, Close, Volume) data.
- **Generating Buy, Sell, or Hold signals** based on predefined strategy conditions.
- **Executing virtual trades** whenever a valid trading signal is generated.
- **Managing open positions** by tracking entry price, exit price, quantity, and current market value.

- **Calculating real-time Profit and Loss (PnL)** and updating portfolio performance continuously.
- **Recording trade history and portfolio statistics** for performance evaluation and future analysis.

Paper trading enables users to evaluate trading strategies under real market conditions without risking actual capital. The platform provides real-time visualizations, including live candlestick charts, portfolio PnL, open positions, trade history, and trading statistics, allowing users to monitor strategy performance before deploying it in the live market.

5.8 AI Strategy Recommendation Methodology

The AI Strategy Recommendation module assists users in selecting the most appropriate trading strategy by analyzing current market conditions and technical indicators. Instead of manually choosing a strategy, the system evaluates market behavior and recommends the strategy that is most likely to perform effectively. The methodology consists of the following steps:

- **Collecting current and historical market data** for the selected financial instrument.
- **Analyzing market conditions** using technical indicators such as moving averages, RSI, MACD, volatility, and price trends.
- **Identifying the prevailing market trend**, such as bullish, bearish, or sideways movement.
- **Comparing market characteristics** with predefined conditions of available trading strategies.
- **Recommending the most suitable trading strategy** based on the analyzed market conditions.
- **Displaying the recommended strategy** along with supporting market insights to assist user decision-making.
- **Allowing users to validate the recommendation** through the backtesting and paper trading modules before real deployment.

The AI Strategy Recommendation module helps users make informed trading decisions by reducing manual analysis and selecting strategies based on data-driven market evaluation. The platform provides intelligent recommendations that improve trading efficiency, support better strategy selection, and assist both beginner and experienced traders in adapting to changing market conditions.

5.9 Performance Metrics

To evaluate strategies, the following metrics are used:

- **Total Profit/Loss** – Net gain or loss over the backtesting period
- **Win/Loss Ratio** – Ratio of profitable trades to losing trades
- **Return on Investment (ROI)** – Profit relative to capital invested
- **Drawdown** – Maximum loss from peak to trough
- **Sharpe Ratio (optional)** – Risk-adjusted return metric

These metrics allow users to assess the effectiveness and risk of each strategy objectively.

5.10 Advantages of Implemented Strategies

The implemented strategies provide:

- Consistent application of rules without human bias
- Quick analysis of large datasets
- Ability to simulate multiple scenarios through backtesting
- Educational insights for beginners on how trading logic works

5.11 Limitations

- Strategies are simplified and may not account for real-world market complexities
- High-frequency trading and live market execution are not included

- Results depend on historical data quality
- External market factors, news, and events are not considered

Despite limitations, the platform serves as a strong educational and analytical tool.

5.12 Summary

This chapter discussed the algorithms and trading strategies used in the Algorithmic Trading Web Platform. It covered trend-following, mean reversion, and arbitrage strategies, along with pseudocode, execution flow, backtesting methodology, and performance metrics. Understanding these algorithms provides insight into automated trading systems and sets the stage for implementation and result analysis in subsequent chapters.

Flow of Calculation

Step 1: Tracking movement (How much is it going up or down)
 (Today - close price - Yesterday - close price)

$$\text{Move Percent age} = \left[\frac{\text{Today - close price} - \text{Yesterday - close price}}{\text{Yesterday - close price}} \times 100 \right]$$

Ex:- Yesterday price = 100 $\frac{110 - 100}{100} \times 100 = 10\%$
 Today price = 110

Step 2: Checking the value of the company which is showing adequate movement
 RSI indicator :- > 70 : overbought (overvalued, risk of pullback)
 < 30 : oversold (undervalued, chance of bounce)

Step 3: Calculation of how much to sell or buy.
 Value to sell or buy : [% change $\times$ 1000]
 ↳ Available for tuning

** Don't take trade
 → Undervalue stock when go up
 → Overvalue stock coming down

Step 4: Calculation how much undervalued and overvalued

Good Value : Buy / Sell	$\left[\begin{array}{l} 2\% \rightarrow 2000 \\ 3\% \rightarrow 3000 \\ 4\% \rightarrow 4000 \\ 5\% \rightarrow 5000 \\ 6\% \rightarrow 6000 \\ 7\% \rightarrow 7000 \end{array} \right]$	Undervalue (x%) (Sell) :	$\left[\begin{array}{l} 2\% - 1\% \rightarrow 2000 \\ 3\% - 1\% \rightarrow 3000 \\ 4\% - 1\% \rightarrow 4000 \\ 5\% - 1\% \rightarrow 5000 \\ 6\% - 1\% \rightarrow 6000 \end{array} \right]$
----------------------------	--	-----------------------------	--

Ex:- If share is 2% undervalued then we will sell share worth of 2000 at 1.6% change

Fig 06 hand Written Risk Calculation

Chapter 6

Methodology

6.1 Introduction

Methodology is one of the most important phases in software development as it describes the practical implementation of the proposed system. It bridges the gap between the conceptual design presented in the previous chapters and the actual working application. This chapter explains the technologies, implementation techniques, software architecture, and development approach used to build the Algorithmic Trading Web Platform (Penn).

The proposed platform has been developed as a modern full-stack web application capable of performing historical backtesting, live market monitoring, intelligent trading strategy recommendation, and interactive financial data visualization. The system integrates multiple technologies to ensure scalability, security, performance, and ease of use.

Unlike the initial prototype that focused only on historical backtesting, the final implementation consists of multiple interconnected modules including a responsive frontend, RESTful backend services, cloud-hosted database, machine learning recommendation engine, market data integration, authentication system, and visualization dashboard. Each module has been designed independently and later integrated to form a unified application.

The frontend of the application has been implemented using Next.js and TypeScript, providing users with an intuitive interface for strategy selection, portfolio analysis, historical backtesting, machine learning recommendations, and live market monitoring. The backend is developed using the Flask framework in Python, which manages business logic, authentication, strategy execution, machine learning processing, and communication with external APIs and the database.

Historical stock market data required for technical analysis and strategy evaluation is obtained through the Yahoo Finance API, while live market information is retrieved using the Fyers API. Persistent application data including user accounts, trading history, portfolios, strategy configurations, and backtesting results are securely stored in a PostgreSQL database hosted on Supabase.

A significant enhancement introduced in the final implementation is the integration of a Machine Learning Recommendation Engine. Instead of recommending trading strategies solely based on individual indicators, the system evaluates the historical performance of multiple algorithmic strategies using several financial metrics and utilizes a Random Forest model to recommend the most suitable strategy for the selected stock. This intelligent recommendation assists users in selecting strategies that best match the historical characteristics of the chosen financial instrument.

The implementation follows a modular development methodology where each component of the system performs a dedicated responsibility while communicating through well-defined REST APIs. Such an architecture improves maintainability, scalability, and future extensibility of the platform.

6.2 Tools and Technologies Used

The development of the Algorithmic Trading Web Platform required the integration of multiple software technologies, programming frameworks, cloud services, and financial libraries. Each technology was selected based on its reliability, scalability, ease of integration, and suitability for financial data processing.

Python serves as the primary programming language for implementing the backend logic of the application. It provides extensive support for financial computing through libraries such as Pandas, NumPy, and Scikit-learn. The flexibility and simplicity of Python make it highly suitable for implementing trading strategies, data analysis, and machine learning models.

Next.js and TypeScript

The user interface has been developed using Next.js with TypeScript. Next.js provides an efficient framework for developing modern web applications with optimized routing and component-based architecture. TypeScript enhances code reliability by introducing static type checking, reducing runtime errors, and improving maintainability.

Flask

Flask has been used to implement the backend server and RESTful APIs. It acts as the communication layer between the frontend, database, external financial APIs, and machine learning engine. Flask processes client requests, executes trading strategies, performs historical backtesting, generates recommendations, and returns processed information in JSON format.

PostgreSQL (Supabase)

The application utilizes PostgreSQL hosted on Supabase as its primary database. The database stores user information, authentication details, portfolio records, strategy configurations, trading history, and historical backtesting results. Supabase provides cloud-based accessibility, improved scalability, and secure data management.

Yahoo Finance API

Yahoo Finance is used to retrieve historical stock market data including Open, High, Low, Close, Adjusted Close, and Volume (OHLCV). This data forms the basis for technical indicator calculation, historical backtesting, and machine learning feature generation.

Fyers API

The Fyers API provides real-time market information for selected stocks. The live market module displays continuously updated price information, allowing users to monitor current market conditions alongside historical analysis.

Plotly

Interactive visualization is implemented using Plotly. The library enables the generation of responsive candlestick charts, buy/sell signal markers, strategy comparison graphs, and other analytical visualizations that assist users in interpreting market behaviour.

Scikit-learn

The machine learning recommendation engine has been implemented using the Scikit-learn library. A Random Forest model analyzes the historical performance metrics of multiple trading strategies and recommends the most suitable strategy based on the characteristics of the selected stock.

6.3 Methodology Overview

The implementation of the proposed system follows a modular and iterative development methodology. Rather than developing the application as a single unit, individual modules were implemented independently, tested separately, and later integrated into a unified trading platform. This approach simplified development, improved code maintainability, and allowed continuous enhancement throughout the project lifecycle.

The implementation methodology consists of the following phases:

1. Requirement Analysis

The first phase involved identifying the objectives and functional requirements of the proposed platform. The system was designed to provide secure user authentication, historical market analysis, live market monitoring, strategy execution, intelligent recommendation, and interactive visualization.

2. System Design

After requirement analysis, the overall architecture of the application was designed by dividing the system into multiple independent modules including the frontend, backend, database, external APIs, strategy engine, machine learning engine, and visualization components.

3. Frontend Development

The frontend interface was developed using Next.js and TypeScript. Separate pages and reusable React components were implemented for user registration, login, dashboard, live market monitoring, historical backtesting, strategy recommendation, portfolio management,

and profile settings. The frontend communicates with the backend through RESTful APIs and dynamically updates user information based on server responses.

4. Backend Development

The Flask backend manages all business logic of the application. It receives requests from the frontend, validates user authentication using JWT tokens, communicates with external APIs, processes market data, executes trading strategies, performs backtesting, interacts with the database, and returns structured responses to the client.

5. Database Implementation

A PostgreSQL database hosted on Supabase stores all application data including user profiles, authentication credentials, portfolio information, strategy configurations, trading records, and historical backtesting results. Database operations are performed securely through backend APIs to maintain data consistency and integrity.

6. Market Data Collection

Historical stock market data is collected through Yahoo Finance, while live market information is obtained using the Fyers API. The retrieved datasets undergo preprocessing, including missing value handling, date formatting, and calculation of technical indicators required by various trading strategies.

7. Trading Strategy Execution

The trading engine implements twenty-three different algorithmic trading strategies covering trend following, momentum trading, breakout strategies, mean reversion, volatility analysis, statistical arbitrage, volume analysis, and AI ensemble techniques. Each strategy independently analyzes the processed market data and generates buy, sell, or hold signals according to predefined trading rules.

8. Machine Learning Recommendation

Performance metrics generated from historical backtesting are evaluated using a scoring mechanism and supplied to a Random Forest model. The machine learning engine analyzes these features and recommends the most suitable trading strategy for the selected stock while

also presenting the reasoning behind the recommendation through explainable performance metrics.

9. Testing and Integration

After independent testing of all modules, the frontend, backend, database, external APIs, trading engine, visualization module, and machine learning engine were integrated into a single web platform. Functional testing and integration testing were conducted to ensure smooth communication between all components and reliable system performance.

6.4 Data Collection and Preparation

The effectiveness of any algorithmic trading system depends heavily on the quality and accuracy of the market data used for analysis. The proposed platform collects both historical and real-time stock market information from reliable financial data providers. Historical data forms the foundation for strategy execution, technical indicator calculation, historical backtesting, and machine learning analysis, while live market data enables users to monitor current market conditions through the web interface.

The platform utilizes the Yahoo Finance API to retrieve historical market information. The downloaded dataset consists of Open, High, Low, Close, Adjusted Close, and Volume (OHLCV) values for the selected stock over a user-defined period. This information is processed before being supplied to the trading engine.

To provide live market monitoring, the system integrates the Fyers API, which continuously supplies updated stock prices and market information. Although the present implementation focuses on market visualization rather than automated trade execution, the integration establishes a strong foundation for future live trading capabilities.

Before the market data is supplied to the trading algorithms, several preprocessing operations are performed to improve consistency and reliability. Missing values and duplicate records are removed to eliminate inconsistencies that could affect trading decisions. Date and time values are converted into a standardized format to ensure compatibility across different modules. The processed dataset is then enriched by calculating multiple technical indicators required by different trading strategies.

The preprocessing stage includes the calculation of moving averages, exponential moving averages, Relative Strength Index (RSI), Moving Average Convergence Divergence (MACD), Bollinger Bands, Average True Range (ATR), Volume Weighted Average Price (VWAP), Rate of Change (ROC), Stochastic Oscillator, Average Directional Index (ADX), and several additional indicators. These calculated features serve as inputs to the strategy engine and the machine learning recommendation model.

After preprocessing, the cleaned dataset is forwarded to the trading strategy engine, where each strategy independently evaluates market conditions and generates trading signals based on its respective trading logic.

6.5 Implementation of Trading Algorithms

The core functionality of the proposed platform is centered around its algorithmic trading engine. This module has been implemented using Python and follows a modular architecture where each trading strategy is developed as an independent component. Such an implementation allows new strategies to be added or existing strategies to be modified without affecting the remaining modules of the application.

Unlike traditional trading applications that rely on a limited number of strategies, the developed platform currently supports twenty-three algorithmic trading strategies. These strategies represent different schools of technical analysis and provide users with diverse approaches to market evaluation. Every strategy processes the same historical market dataset but generates trading signals according to its own mathematical rules and technical indicators.

6.6 Machine Learning Recommendation Engine

One of the major enhancements introduced in the final implementation of the proposed platform is the incorporation of a Machine Learning Recommendation Engine. Rather than requiring users to manually compare multiple trading strategies, the recommendation engine

automatically analyzes strategy performance and recommends the most suitable trading strategy for the selected stock.

The recommendation process begins after all implemented trading strategies complete their historical backtesting. For every strategy, the system computes several financial performance indicators that describe both profitability and associated trading risk. These indicators collectively represent the historical behavior of each strategy and are used as features for machine learning prediction.

The current implementation evaluates each strategy using the following performance parameters:

Total Return, Return Percentage, Number of Executed Trades, Winning Trades, Losing Trades, Win Rate, Average Profit, Average Loss, Profit Factor, Maximum Drawdown, Sharpe Ratio, Volatility, Risk Level, Overall Performance Score

These parameters are calculated automatically after every backtesting session and are stored for further analysis.

To improve recommendation quality, the platform introduces a scoring mechanism that assigns an overall score to every trading strategy. The scoring algorithm considers profitability, consistency, trading frequency, risk exposure, and market stability before generating the final score. Strategies demonstrating strong historical performance while maintaining acceptable levels of risk receive higher scores than strategies producing unstable or inconsistent returns.

The generated performance metrics and calculated scores are supplied to a Random Forest Machine Learning model implemented using the Scikit-learn library. Random Forest was selected because of its ability to analyze multiple performance indicators simultaneously, its robustness against overfitting, and its high predictive capability for structured financial datasets.

Based on the learned relationships between strategy performance and market characteristics, the model predicts the most appropriate trading strategy for the selected stock. The recommended strategy is then presented to the user together with additional supporting

information, including historical return, risk level, Sharpe ratio, win rate, total trades, and overall performance score.

To improve transparency and user confidence, the recommendation engine also provides explainable outputs by displaying the major factors contributing to the recommendation. Instead of functioning as a black-box prediction system, users are able to understand why a particular strategy has been recommended by examining the underlying financial metrics. This approach assists both beginner and experienced traders in making more informed decisions.

The recommendation engine is designed to operate independently of the frontend interface, allowing future machine learning models or additional financial features to be incorporated without modifying other modules of the application.

6.7 Backtesting and Performance Evaluation

Backtesting is an essential component of the proposed Algorithmic Trading Web Platform as it enables trading strategies to be evaluated using historical market data before any real-world application. The implemented backtesting engine simulates historical trades by executing the generated buy and sell signals on past market prices without exposing users to financial risk.

For each selected stock, historical market data is downloaded and supplied to the chosen trading strategy. The strategy analyzes the dataset, calculates technical indicators, and generates trading signals whenever predefined market conditions are satisfied. These signals are then processed by the backtesting engine, which simulates trade execution while maintaining records of entry price, exit price, trade duration, and cumulative portfolio value.

During simulation, the engine continuously monitors trading positions, calculates realized profits and losses, and updates the virtual portfolio after every completed transaction. Once the historical simulation is completed, several performance metrics are automatically calculated to evaluate the effectiveness of the selected strategy.

The implemented platform computes important performance indicators including total return, percentage return, number of executed trades, winning trades, losing trades, win rate, average profit, average loss, maximum drawdown, Sharpe ratio, profit factor, volatility, and estimated

risk level. These metrics provide users with a comprehensive understanding of the strengths and limitations of each trading strategy under historical market conditions.

The calculated performance indicators are subsequently forwarded to the Machine Learning Recommendation Engine, where they contribute to the overall strategy scoring process. In addition, selected backtesting results are stored in the PostgreSQL database for future reference and comparison.

The modular implementation of the backtesting engine allows it to evaluate any newly introduced trading strategy without requiring modifications to the remaining system components. This extensible design supports future expansion of the platform while maintaining consistency across all strategy evaluations.

6.8 User Interface Implementation

A user-friendly interface is essential for enabling users to interact efficiently with the algorithmic trading platform. The frontend of the proposed system has been developed using Next.js with TypeScript, providing a responsive, scalable, and modern web application capable of supporting multiple trading-related functionalities.

The application follows a component-based architecture in which different pages and reusable UI components work together to provide a seamless user experience. Users can navigate through various modules including authentication, dashboard, historical backtesting, machine learning recommendations, live market monitoring, portfolio management, and profile settings.

The login and registration pages provide secure user authentication through JSON Web Token (JWT) based authorization. After successful authentication, users are redirected to the dashboard where they can access the major functionalities of the application.

The historical backtesting interface allows users to select a stock symbol, choose one of the implemented trading strategies, specify the desired time period, and initiate historical analysis. Once processing is completed, the generated results are displayed using interactive tables and graphical representations.

The recommendation module provides an intelligent interface where users can evaluate multiple trading strategies simultaneously. After processing historical market data, the recommendation engine displays the highest-ranked strategy together with supporting financial metrics, allowing users to understand the reasoning behind the recommendation.

The live market dashboard communicates with the backend to retrieve continuously updated stock market information obtained through the Fyers API. Users can monitor current market prices while simultaneously accessing historical analytical information generated by the platform.

The frontend also includes portfolio management and profile pages, allowing users to manage personal information and view stored trading-related records retrieved from the PostgreSQL database. The responsive design ensures that the interface remains consistent across different screen sizes and provides an intuitive experience for both beginner and experienced users.

6.9 Integration of Modules

The proposed Algorithmic Trading Web Platform has been developed using a modular architecture in which each subsystem performs a dedicated responsibility while maintaining communication with other modules through RESTful APIs. This architecture improves scalability, maintainability, and simplifies future system enhancements.

The workflow begins when the user submits a request through the frontend interface. Depending on the requested operation, the frontend communicates with the Flask backend using REST APIs. The backend validates authentication, retrieves market data, executes trading strategies, performs historical backtesting, generates machine learning recommendations, and interacts with the PostgreSQL database whenever persistent storage is required.

The processed information is converted into structured JSON responses and transmitted back to the frontend. Finally, the frontend presents the information using interactive Plotly visualizations, performance tables, recommendation cards, and analytical dashboards.

This modular communication architecture ensures loose coupling between different components while enabling each module to evolve independently without affecting the overall functionality of the platform.

6.10 Testing and Validation

Comprehensive testing was performed throughout the development process to verify the correctness, reliability, and stability of the proposed system. Individual modules were tested independently before integrating them into the complete application.

Unit Testing was conducted on individual backend modules including authentication, database operations, trading strategy execution, machine learning recommendation, and API communication. Each module was verified to ensure that it produced the expected output for different input scenarios.

Integration Testing was performed after combining the frontend, backend, external APIs, and database. The objective was to verify that information flowed correctly between all system components and that API responses were accurately reflected within the user interface.

Functional Testing ensured that every feature implemented in the application operated according to the specified requirements. This included user registration, login, historical backtesting, strategy execution, recommendation generation, market data retrieval, portfolio management, and visualization of analytical results.

The communication between the frontend and backend was tested by sending multiple API requests under different operating conditions. Responses were verified to ensure successful authentication, correct strategy execution, proper database interaction, and accurate display of information.

The historical backtesting engine was validated by executing multiple trading strategies on different stocks and comparing generated trading signals with expected technical indicator behavior. Similarly, the recommendation engine was tested by verifying that strategy rankings were generated successfully based on calculated performance metrics. The database layer was validated by ensuring successful insertion, retrieval, updating, and deletion of user

information, portfolios, backtesting records, and recommendation data within the PostgreSQL database hosted on Supabase.

Error handling mechanisms were also incorporated throughout the application to improve system robustness. Invalid user inputs, authentication failures, unavailable market data, API communication errors, and database exceptions were handled gracefully through appropriate validation and error messages. The testing process confirmed that the developed platform performs reliably under normal operating conditions and provides consistent analytical outputs across different modules.

6.11 Summary

This chapter presented the implementation methodology adopted for developing the proposed Algorithmic Trading Web Platform. The system was implemented using a modern full-stack architecture consisting of a Next.js and TypeScript frontend, a Flask-based backend, PostgreSQL database hosted on Supabase, Yahoo Finance and Fyers APIs for market data acquisition, Plotly for interactive visualization, and a Random Forest-based machine learning recommendation engine.

The implementation process included market data acquisition, preprocessing, execution of twenty-three algorithmic trading strategies, historical backtesting, intelligent strategy recommendation, secure user authentication, database management, and integration of all modules through RESTful APIs. The modular architecture adopted during development improves scalability, maintainability, and facilitates future enhancements.

Comprehensive testing and validation ensured reliable communication between system components and verified the correctness of implemented functionalities. The successful integration of data processing, trading algorithms, machine learning, and visualization resulted in a robust web-based platform capable of assisting users in market analysis and strategy evaluation.

The outcomes of the implemented system, including performance analysis, experimental observations, and discussion of results, are presented in the next chapter.

Chapter 7

Results and Discussion

7.1 Overview of Experimental Setup

The experimental setup for this project was designed to evaluate the performance and effectiveness of the proposed algorithmic trading web platform under controlled conditions. The system was tested using historical stock market data, which enables objective assessment of trading strategies through backtesting without financial risk. Historical price data, including open, high, low, close, and volume values, was collected from reliable market data sources for selected stocks over a defined time period. A fixed initial capital was assumed for all experiments to ensure consistency and fairness in performance comparison. Rule-based trading strategies utilizing technical indicators were applied to the dataset to generate buy and sell signals.

Here is an example of how it works:

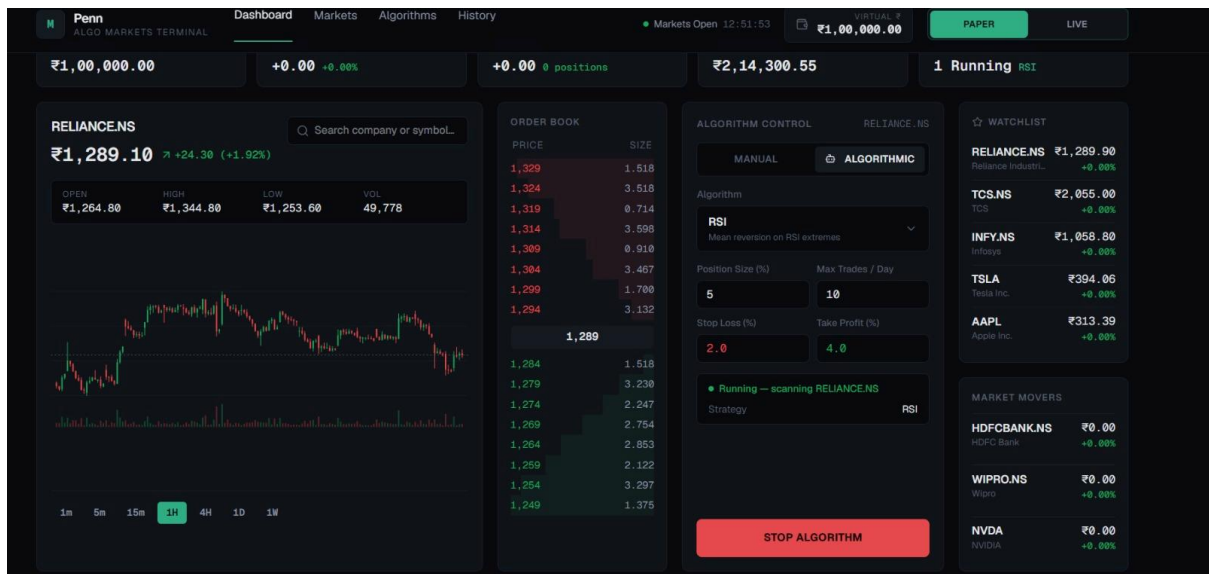


Fig 7 Paper Trading/ Live Trading setup

7.2 Backtesting Results

The backtesting results obtained from the proposed algorithmic trading system demonstrate the effectiveness of the implemented trading strategies when applied to historical market data. The backtesting process simulated real trading conditions by

executing trades based on predefined strategy rules without using real capital. During the experiment, the system successfully generated buy and sell signals and executed virtual trades across the selected dataset. Key performance indicators such as total number of trades, profit and loss, and trade success rate were recorded for analysis. The results indicate that the system was able to identify favorable market conditions and execute trades accordingly.

The backtesting outcomes show a noticeable improvement in trading consistency compared to manual trading approaches. The automated execution ensured timely entry and exit points, reducing delays caused by human intervention. The generated trade logs provided detailed information on each transaction, including entry price, exit price, and individual trade returns.

Overall, the backtesting results validate the functional correctness of the proposed system and confirm its ability to evaluate trading strategies effectively. These results serve as a foundation for further performance analysis and graphical interpretation discussed in subsequent sections.

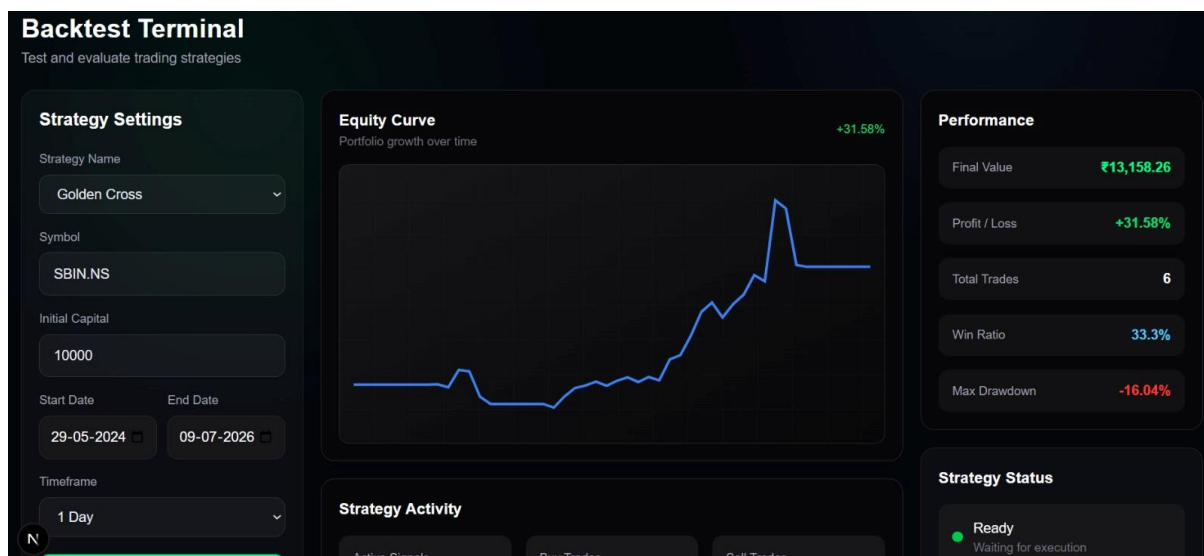


Fig 8 Backtesting setup

7.3 Live Trading (Paper Trading) Results

The live trading (paper trading) results demonstrate the practical implementation of the proposed algorithmic trading system in a simulated real-time market environment. Unlike backtesting, where historical data is used for strategy evaluation, the paper trading module continuously fetches live market data and generates trading decisions without involving

actual financial transactions. This enables users to assess the real-time performance of trading strategies while eliminating financial risk.

During the experiment, the system successfully monitored live market prices, analyzed incoming OHLCV (Open, High, Low, Close, Volume) data at regular intervals, and generated Buy, Sell, or Hold signals based on the selected trading strategy. Whenever a valid trading signal was generated, the paper trading engine executed virtual orders, maintained open positions, and updated the trade history automatically.

The paper trading system continuously calculated unrealized Profit and Loss (PnL), tracked open positions, recorded completed trades, and maintained important trading statistics such as total trades executed, winning trades, losing trades, and overall portfolio performance. All trading activities were displayed through an interactive dashboard, allowing users to monitor strategy execution in real time.

The results indicate that the proposed system successfully replicates the workflow of a real algorithmic trading platform while ensuring that no actual capital is involved. The implementation validates the correctness of real-time data acquisition, signal generation, virtual trade execution, position management, and portfolio monitoring. Consequently, the paper trading module serves as an effective bridge between historical backtesting and real-world live trading, allowing users to validate trading strategies under actual market conditions before deploying them with real funds.

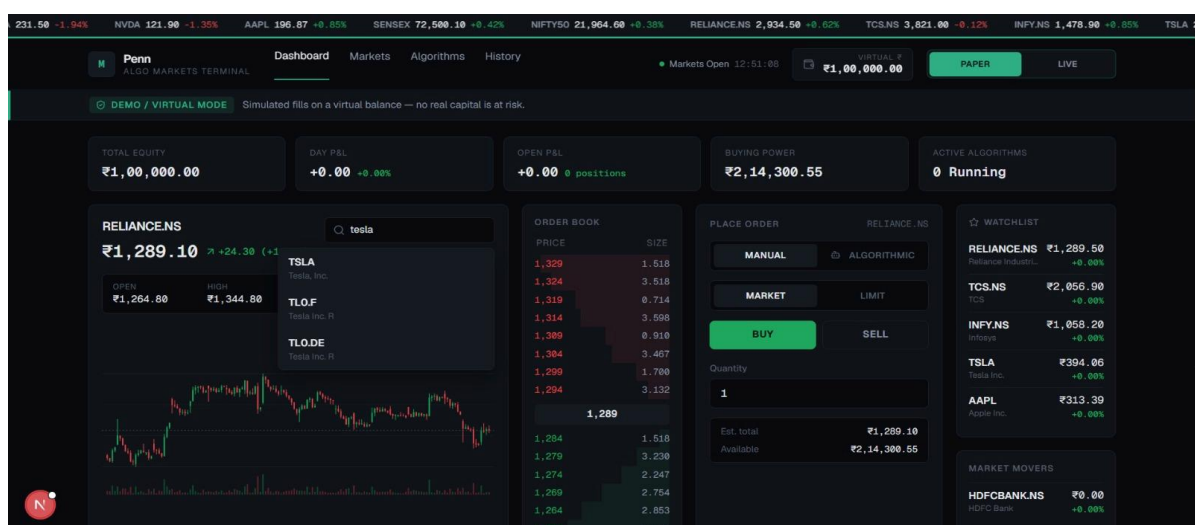


Fig 9 Paper Trading Results with built-in search recommendation

STRATEGY	SCORE	RETURN	WIN RATE	SHARPE	MAX DD	TRADES	RISK
Bollinger Bands	56	+90.82%	83.3%	0.93	-38.65%	12	High
ZScore Reversion	56	+90.82%	83.3%	0.93	-38.65%	12	High
RSI	55.8	+86.34%	83.3%	0.91	-38.65%	12	High
Stochastic	36.4	+17.79%	81.8%	0.4	-48.12%	22	High
Bollinger Squeeze	34.7	+12.95%	100%	0.4	-16.5%	2	High
VWAP Bounce	0	-1.82%	11.1%	0.15	-29.93%	18	High

Fig 10 Live Trading Results (Demo) with built-in search recommendation

7.4 AI Strategy Recommendation Results

The AI Strategy Recommendation module enhances the proposed algorithmic trading system by assisting users in selecting the most suitable trading strategy based on current market conditions. Instead of relying solely on manual strategy selection, the system analyzes key market indicators and provides intelligent recommendations that improve decision-making. This feature is particularly beneficial for beginner traders who may not possess extensive knowledge of technical analysis or market behavior.

During the experiment, the AI recommendation engine successfully analyzed market trends, price movements, volatility, and technical indicators before suggesting the most appropriate trading strategy. Based on the analysed data, the system recommended strategies such as Golden Cross, RSI, EMA Cross, MACD Cross, or SuperTrend, depending on the prevailing .

STRATEGY	SCORE	RETURN	WIN RATE	SHARPE	MAX DD	TRADES	RISK
Bollinger Bands	56	+90.82%	83.3%	0.93	-38.65%	12	High
ZScore Reversion	56	+90.82%	83.3%	0.93	-38.65%	12	High
RSI	55.8	+86.34%	83.3%	0.91	-38.65%	12	High
Golden Cross	48.7	+51.65%	60%	0.7	-32.57%	10	High
Price Breakout	47.3	+63.26%	40%	0.8	-25.54%	10	High
Donchian Breakout	47.3	+63.26%	40%	0.8	-25.54%	10	High
MACD Zero Line	46.4	+34.21%	33.3%	0.57	-24.97%	18	High
Momentum Burst	45.7	+42.03%	38.5%	0.64	-34.45%	26	High
MACD Cross	42.7	+41.87%	30.4%	0.64	-35.65%	46	High
Stochastic	36.4	+17.79%	81.8%	0.4	-48.12%	22	High
Bollinger Squeeze	34.7	+12.95%	100%	0.4	-16.5%	2	High

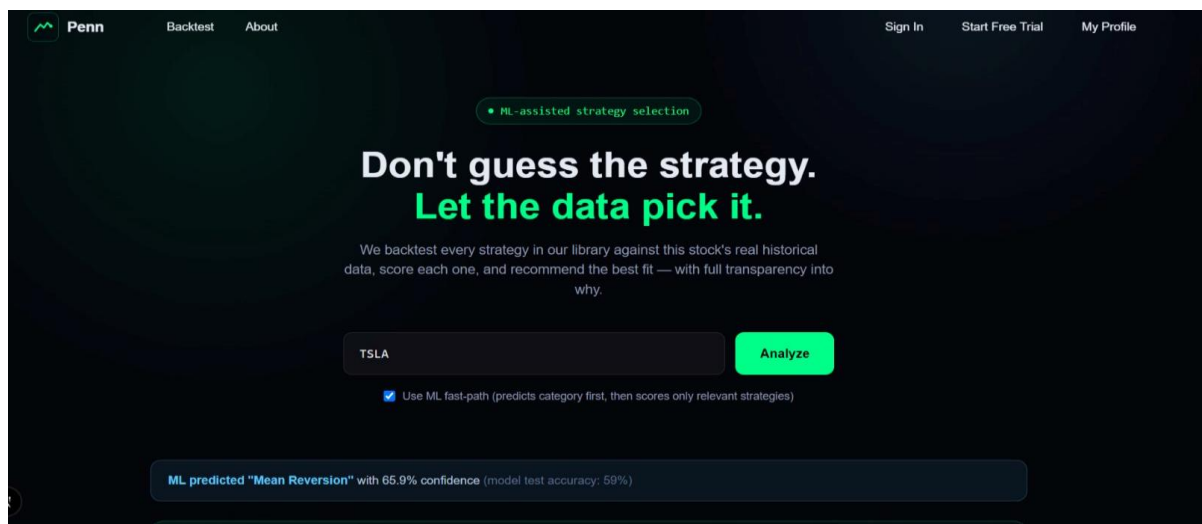


Fig 11 & 12: Recommendation Engine

7.5 User Profile (My Profile) Results

The User Profile (My Profile) module provides a centralized interface for managing user information and monitoring individual trading activities within the proposed algorithmic trading system. This module enables users to securely access their personal details, account information, and trading records through a single dashboard. By maintaining personalized trading data, the system enhances user experience and simplifies portfolio management.

During the implementation, the module successfully displayed essential user information, including profile details, account credentials, and trading preferences. It also maintained a record of the user's trading history, active positions, and portfolio statistics, allowing users to monitor their trading performance over time. The profile dashboard was dynamically updated whenever new trades were executed or account information changed.

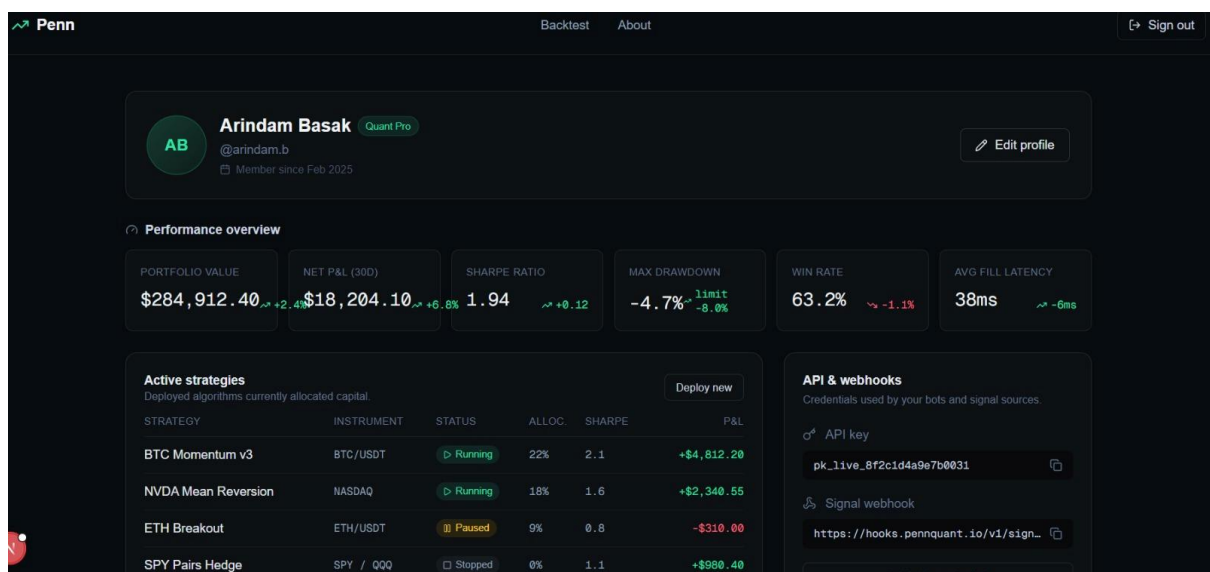


Fig 13 User Page (My Profile)

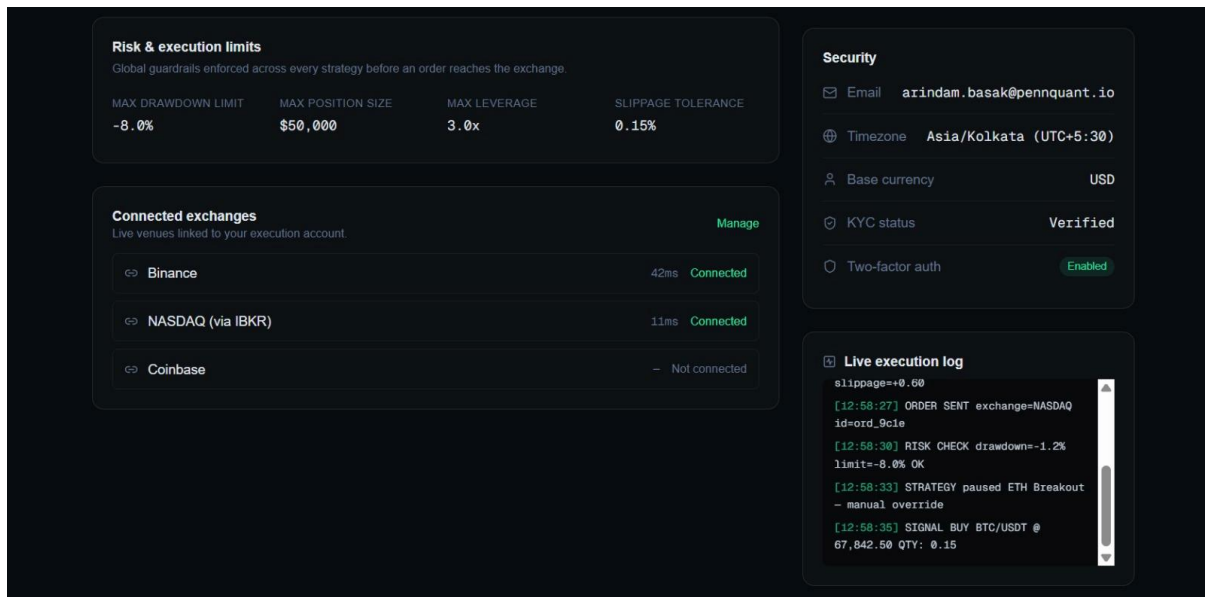


Fig 14 User Page (My Profile)

7.6 Comparison of Existing and Proposed System

A comparative analysis was conducted to evaluate the effectiveness of the proposed algorithmic trading system against existing manual and partially automated trading approaches. The comparison focuses on key parameters such as trading accuracy, execution speed, consistency, and analytical support.

In traditional or manual trading systems, trading decisions are largely influenced by human judgment and emotions, which can lead to inconsistent outcomes and delayed execution. In contrast, the proposed system automates the decision-making process using predefined strategy rules, ensuring consistent and unbiased trade execution.

Existing trading systems often provide limited backtesting and visualization features, making it difficult for users to evaluate strategy performance before deployment. The proposed system addresses this limitation by integrating a comprehensive backtesting framework along with graphical visualizations such as candlestick charts and profit/loss trends, enabling better performance analysis.

Furthermore, professional algorithmic trading platforms are often complex and costly, restricting accessibility for students and retail traders. The proposed web-based platform offers a simplified and cost-effective solution while maintaining essential trading functionalities. Overall, the comparison demonstrates that the proposed system offers improved usability, transparency, and analytical capability compared to existing approach

7.7 Performance Discussion

The performance analysis of the proposed algorithmic trading system indicates that the implemented strategy operates effectively under the tested market conditions. The backtesting results show that automated trade execution based on predefined rules ensures consistency and reduces the impact of emotional decision-making, which is a major limitation of manual trading.

The system demonstrated better performance during stable and trending market conditions, where technical indicators were able to generate clear buy and sell signals. Profitable trades were observed when price movements aligned with the strategy logic, while reduced performance was noticed during periods of high volatility or sudden market reversals. This behavior is consistent with the characteristics of rule-based trading strategies.

The graphical visualizations further supported the numerical results by clearly illustrating trade execution points and profit trends. The equity curve and candlestick charts provided insights into drawdowns and recovery phases, enabling effective assessment of strategy stability and risk exposure. The system's ability to generate detailed trade logs enhanced transparency and result interpretability.

Overall, the performance discussion confirms that the proposed system successfully meets its intended objectives of automation, analysis, and visualization. While the system shows promising results in a simulated environment, its performance may vary under real-time market conditions, highlighting opportunities for further enhancement and optimization.

7.8 Overall Summary of Results

The experimental results obtained from the proposed algorithmic trading system demonstrate that all major modules function successfully and integrate seamlessly to provide a complete trading platform. The system was evaluated using historical market data, simulated live market conditions, and intelligent strategy recommendation techniques, ensuring that each component performed according to the intended objectives.

The **Backtesting module** successfully evaluated multiple trading strategies on historical market data and generated detailed performance metrics such as trade logs, profit and loss,

total returns, and strategy accuracy. This enabled users to assess the effectiveness of trading strategies before deploying them in a live environment.

The **Live Trading (Paper Trading) module** effectively simulated real-time trading by continuously fetching live market data, generating trading signals, executing virtual trades, maintaining open positions, and calculating real-time portfolio performance without involving actual financial transactions. This provided users with a realistic trading experience while eliminating financial risk.

The **AI Strategy Recommendation module** successfully analyzed market conditions and recommended the most suitable trading strategy based on current market trends and technical indicators. This intelligent recommendation system simplified strategy selection and assisted users in making more informed trading decisions.

The **User Profile module** efficiently managed user information, trading history, portfolio statistics, and account details through a secure and user-friendly interface. The centralized dashboard enabled users to monitor their overall trading activities and review their performance over time.

Overall, the proposed system successfully achieves its primary objective of providing an integrated algorithmic trading platform that combines **strategy evaluation, AI-assisted decision-making, simulated live trading, and user portfolio management** within a single application. The obtained results validate the functional correctness, reliability, and practical applicability of the system. Furthermore, the platform offers a safe environment for users to learn, test, and optimize trading strategies before participating in real financial markets, thereby reducing potential trading risks and improving decision-making confidence.

Chapter 8

8.1 Conclusion

This project successfully presented the design and implementation of an **Algorithmic Trading Web Platform** aimed at automating trading decisions and evaluating trading strategies through systematic backtesting. The proposed system was developed to address the limitations of manual trading and existing complex trading platforms by providing a simplified, accessible, and user-friendly solution.

The system integrates historical market data analysis, rule-based trading strategies, and a backtesting engine to generate buy and sell signals in a simulated trading environment. By incorporating graphical visualization tools such as candlestick charts and trade performance metrics, the platform enables users to effectively analyze strategy behavior and profitability. The implementation demonstrates that automated trading can improve consistency, reduce emotional bias, and support informed decision-making.

Overall, the project meets its defined objectives and successfully bridges the gap between theoretical algorithmic trading concepts and practical implementation. The developed platform serves as a reliable foundation for learning, experimentation, and further enhancement in the domain of algorithmic trading and financial technology.

8.2 Scope for Future Work

Although the proposed algorithmic trading platform successfully fulfills its intended objectives, there are several areas where the system can be further enhanced to improve functionality, accuracy, and real-world applicability.

Future work may include the implementation of live trading capabilities through broker APIs, allowing automated order execution in real trading environments. The system can also be expanded to support additional and more advanced trading strategies, including machine learning and artificial intelligence–based models for predictive analysis.

Another potential enhancement is the incorporation of risk management and portfolio optimization techniques, such as stop-loss mechanisms, position sizing, and diversification strategies. Improving system scalability, performance optimization, and multi-asset support would further increase the usability of the platform. Additionally, enhancing the user interface and providing detailed analytics dashboards can improve

user experience and decision support.

These future extensions would transform the proposed system into a more robust, intelligent, and industry-ready algorithmic trading platform, offering broader applications in financial research, education, and professional trading environments.

References & Bibliography

Books

1. Murphy, J. J., *Technical Analysis of the Financial Markets*, New York Institute of Finance, 1999.
2. Chan, E. P., *Algorithmic Trading: Winning Strategies and Their Rationale*, Wiley Trading, 2013.
3. Narang, R. K., *Inside the Black Box: A Simple Guide to Quantitative and High Frequency Trading*, Wiley Finance, 2013.
4. Hull, J. C., *Options, Futures, and Other Derivatives*, Pearson Education, 9th Edition, 2018.
5. Pring, M. J., *Technical Analysis Explained*, McGraw-Hill Education, 2014.

Research Papers / Journals

6. Aldridge, I., “High-Frequency Trading: A Practical Guide to Algorithmic Strategies and Trading Systems,” *Wiley Finance*, 2013.
7. Kumar, M. and Thenmozhi, M., “Forecasting Stock Index Movement: A Comparison of Support Vector Machines and Random Forest,” *Journal of Financial Markets*, 2014.
8. Dixon, M., Klabjan, D., & Bang, J. H., “Classification-Based Financial Markets Prediction using Deep Neural Networks,” *Algorithmic Finance*, 2017.

Web Resources

9. Yahoo Finance – Historical Market Data Documentation.
10. National Stock Exchange (NSE) India – Market Data and Reports.
11. Python Official Documentation – <https://docs.python.org>

- 12. Pandas Documentation – <https://pandas.pydata.org>
- 13. NumPy Documentation – <https://numpy.org>
- 14. Matplotlib Documentation – <https://matplotlib.org>
- 15. Investopedia – Technical Indicators and Trading Concepts.

16. Backtrader Documentation – Python Backtesting Framework.

Quantopian Lectures (Archived) – Algorithmic Trading Concepts.